Sécurité des Systèmes Avancée

TP 3 – Analyse de logiciels malveillants

Rapport d'analyse de l'échantillon SENTECH-TRAINING-SAMPLE

Étudiant	M'BOH Mohamed
Cadre	SOC SenTech – équipe de réponse à incident
Enseignant	Abdoul Haatii SARR
Date	31 octobre 2026
Laboratoire	VMware : Yubuntu DMZ (analyse) + Kali-Lunix (INetSim), réseau host-only VMnet1

1. Verdict et résumé exécutif

Verdict : MALVEILLANT par comportement (échantillon d'entraînement bénin).

L'échantillon sample.bin est un exécutable ELF 64 bits x86-64 de 16 496 octets. À l'exécution il affiche un message, dépose un fichier caché dans /tmp contenant une fausse clé, émet une balise HTTP (GET /gate.php) vers le domaine malware-c2.example, ajoute une entrée @reboot au crontab de l'utilisateur et laisse un processus enfant résident pendant 120 secondes. Il ne détruit ni ne chiffre aucune donnée : c'est un artefact pédagogique qui reproduit les quatre comportements classiques d'un implant (dépôt, balise de commande et contrôle, persistance, processus résident).

Les analyses statique et dynamique concordent. Le hachage n'est pas un indicateur durable (un simple empaquetage UPX le change) ; la détection robuste repose sur les TTP : balise HTTP vers un domaine inhabituel, entrée @reboot pointant vers /tmp, fichier caché dans /tmp et processus orphelin. Une règle YARA testée est fournie. Plan : 2. laboratoire, 3. triage, 4. analyse statique, 5. analyse dynamique, 6. synthèse (IOC, ATT&CK, YARA), 7. recommandations, 8. cas Windows (PE) et bonnes pratiques, 9. limites et écarts.

2. Partie 0 – Laboratoire d'analyse

Le laboratoire isole l'échantillon sur un réseau host-only sans Internet. Un service INetSim simule les services réseau pour capturer la balise de commande et contrôle sans la laisser sortir.

Machine	Rôle	Système	Adresse	Réseau
Yubuntu DMZ	VM d'analyse (héberge l'échantillon)	Ubuntu 24.04.3 LTS	192.168.10.20/24 (ens33)	VMnet1 host-only, aucune route par défaut
Kali-Lunix	VM services (INetSim + DNS)	Kali 6.16.8	192.168.10.10/24 (eth0)	VMnet1 host-only
Hôte Windows	Poste de pilotage (SSH, scp)	Windows	192.168.10.1 (VMnet1)	DHCP de VMnet1 désactivé : IP statiques

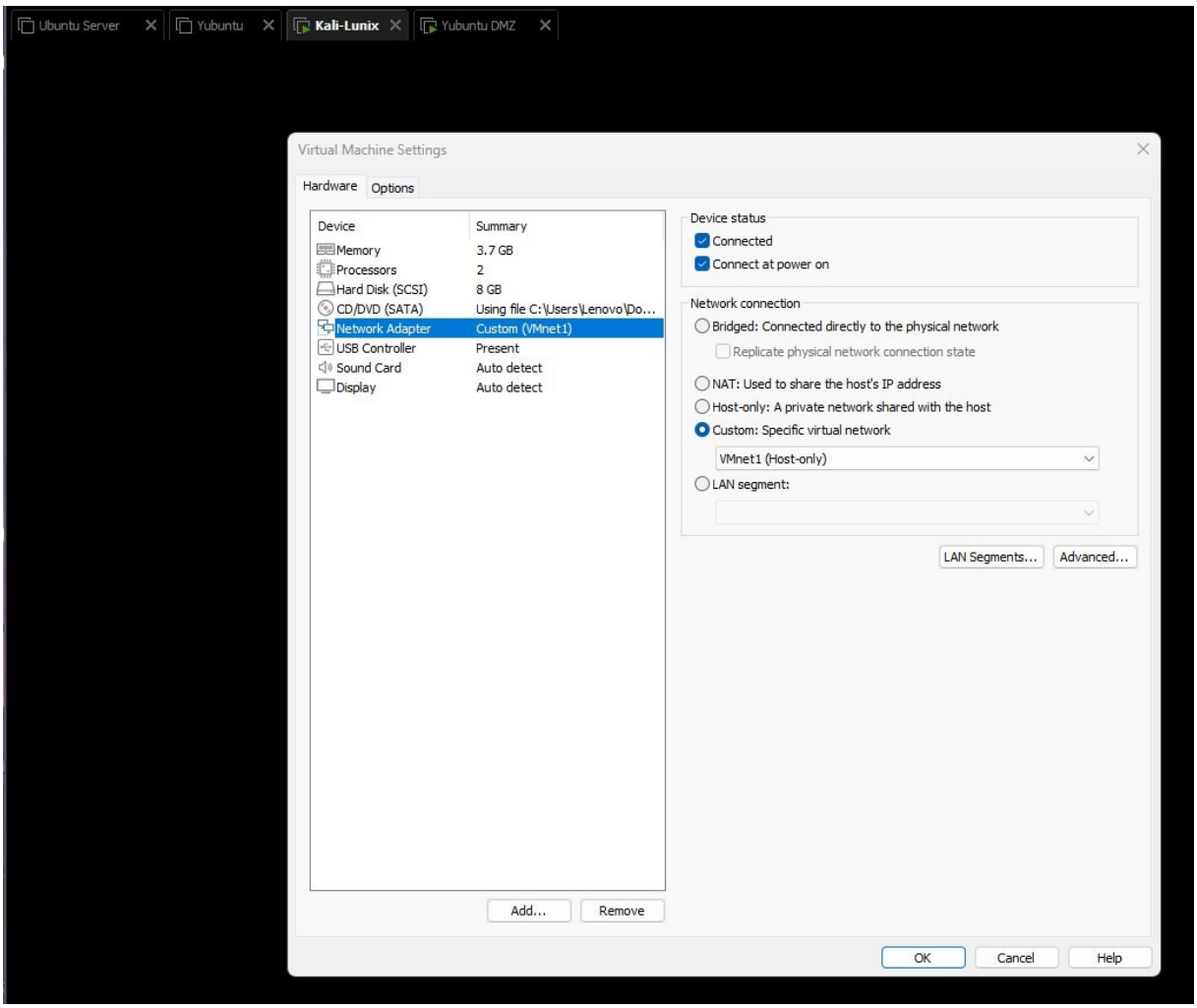


Figure 1 – Réglages VMware de Kali-Lunix : carte en Custom VMnet1 (Host-only)

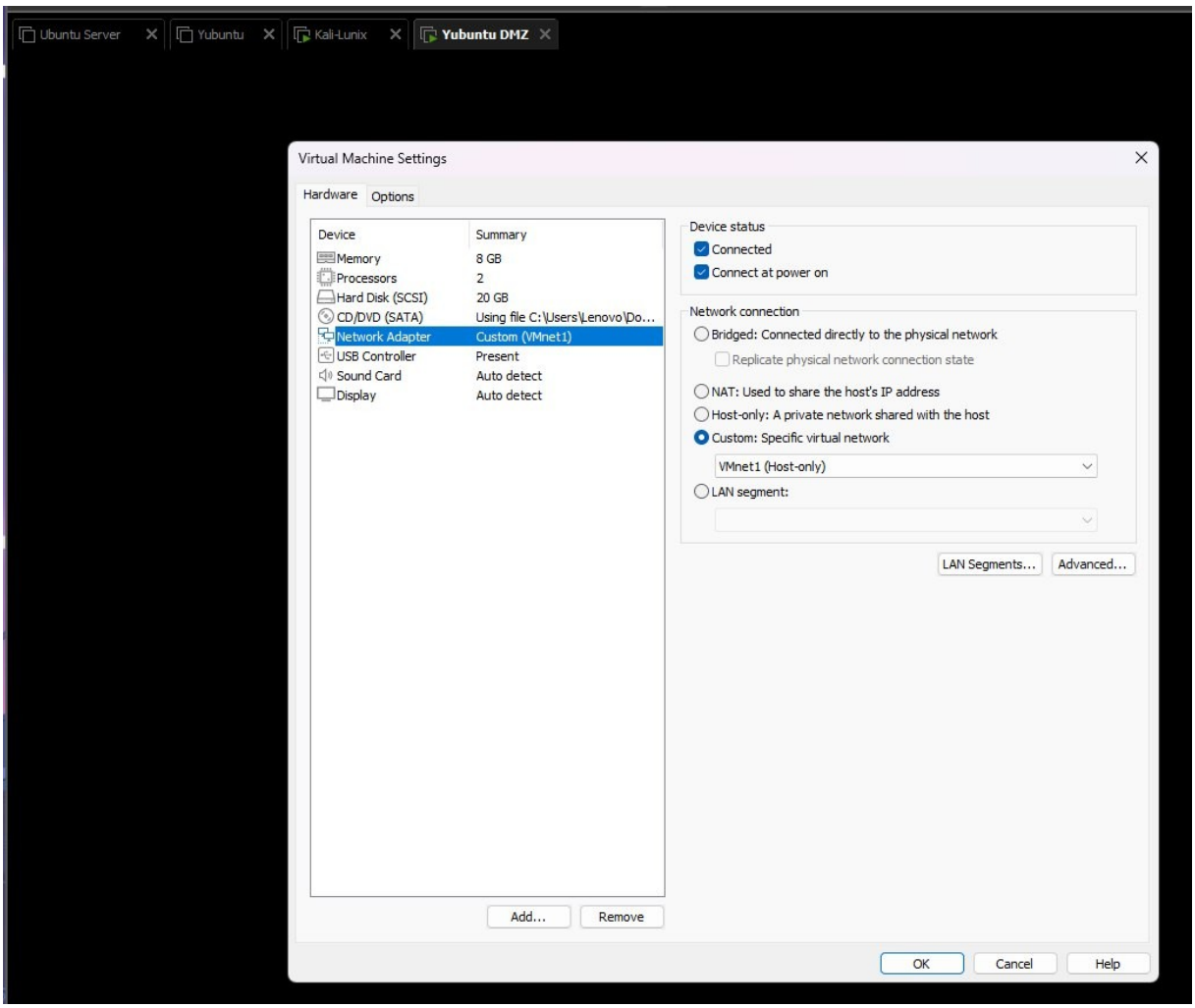


Figure 2 – Réglages VMware de Yubuntu DMZ : carte en Custom VMnet1 (Host-only)

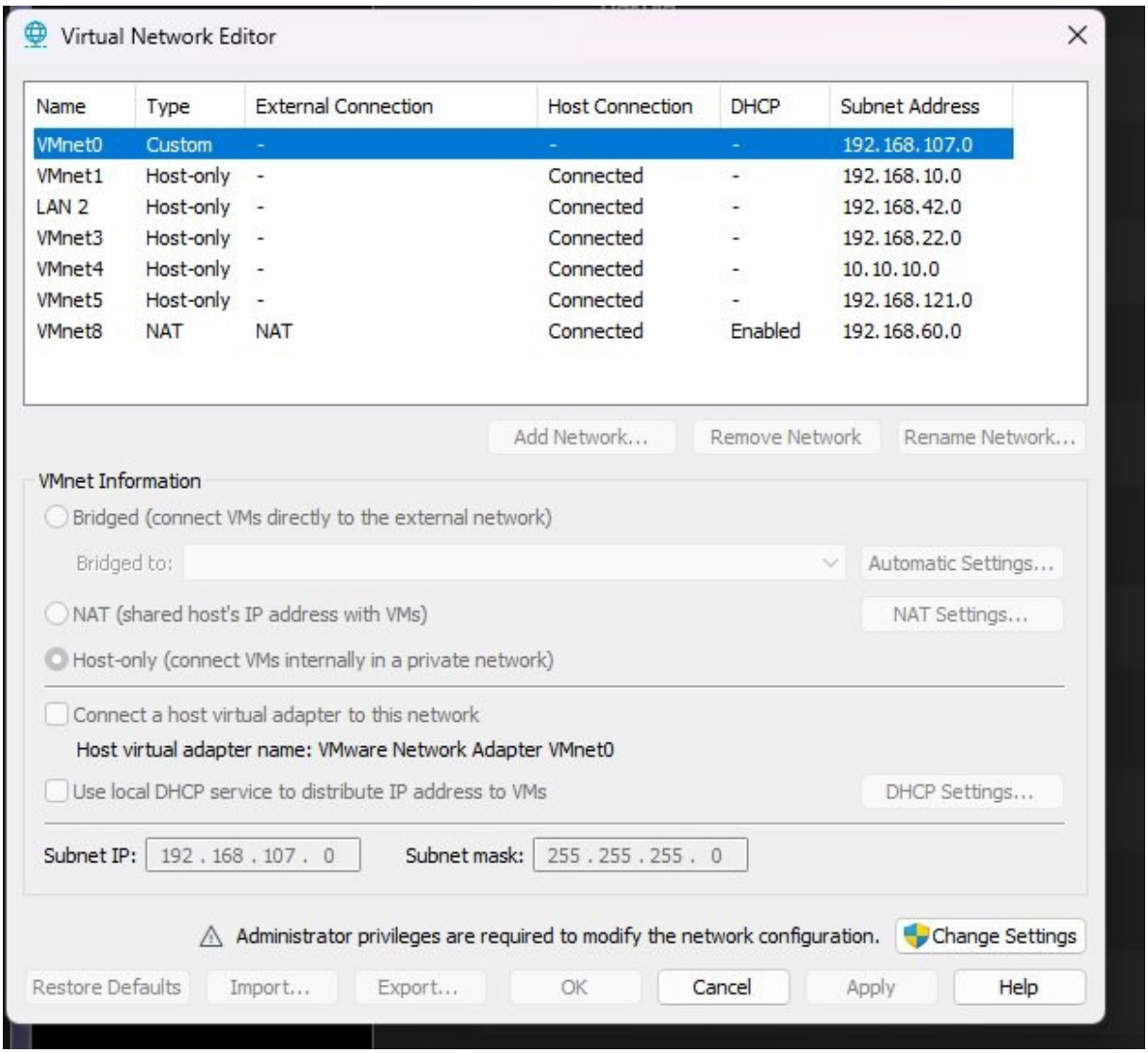


Figure 3 – Virtual Network Editor : VMnet1 en host-only, 192.168.10.0, DHCP désactivé

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Installez la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations ! https://aka.ms/PSWindows

PS C:\Users\Lenovo> ipconfig | Select-String -Context 0,4 "VMnet1"

> Carte Ethernet VMware Network Adapter VMnet1 :

     Suffixe DNS propre à la connexion. . . :
     Adresse IPv6 de liaison locale. . . . .: fe80::6423:9b1e:4b0c:3ef0%28
     Adresse IPv4. . . . . . . . . . . . . .: 192.168.10.1


PS C:\Users\Lenovo>
```

Figure 4 – Côté hôte : carte VMnet1 en 192.168.10.1

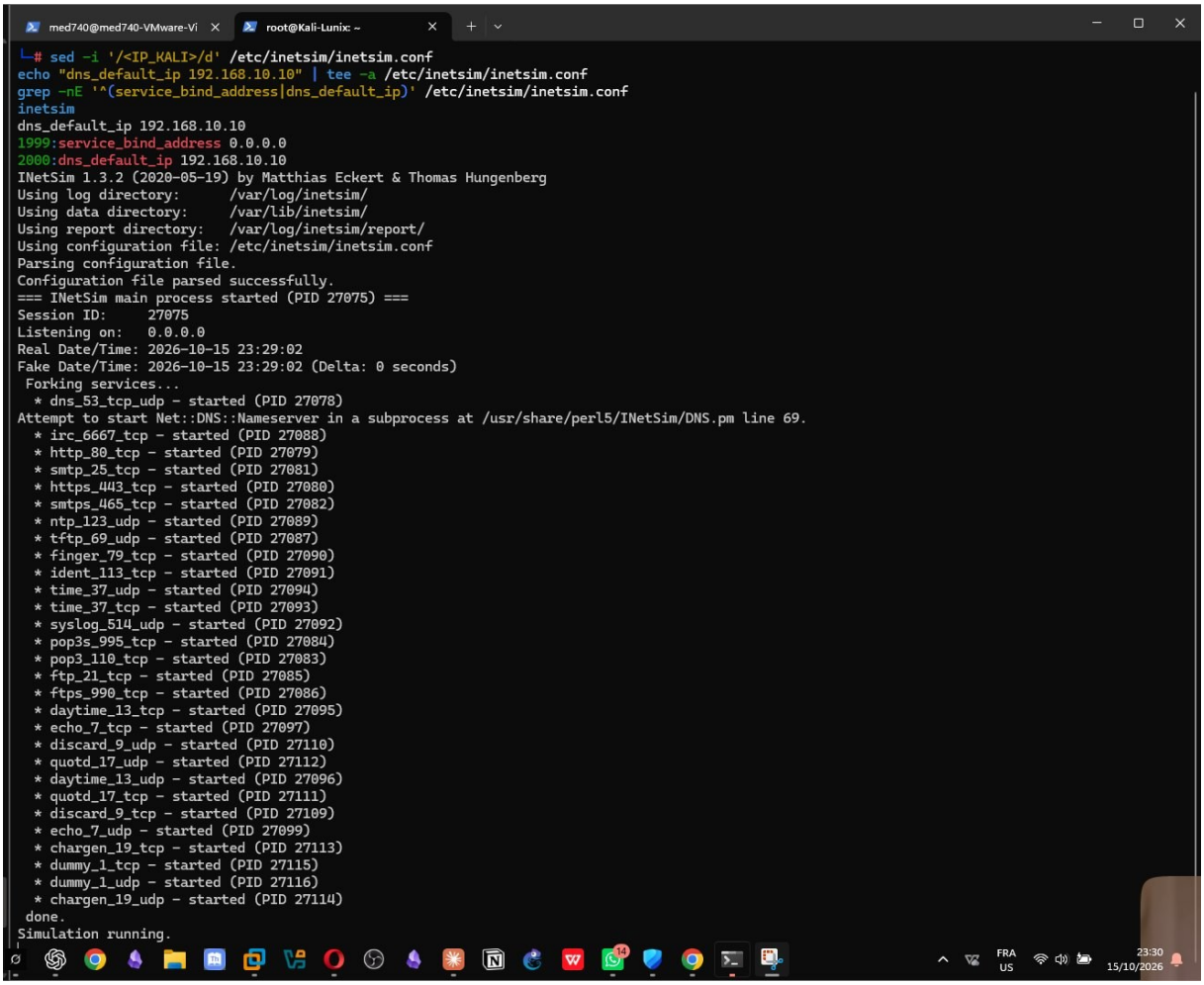


Figure 5 – Configuration d'INetSim (bind 0.0.0.0, dns_default_ip 192.168.10.10) et démarrage

Preuves demandées

- **(a) Le domaine malware-c2.example est servi par INetSim :** getent renvoie 192.168.10.10 et le curl affiche la page factice « INetSim default HTML page » (haut de la capture ci-dessous).
- **(b) Aucune fuite Internet :** ip r ne montre que le réseau 192.168.10.0/24 (pas de route par défaut) et ping 8.8.8.8 échoue (« Le réseau n'est pas accessible »).

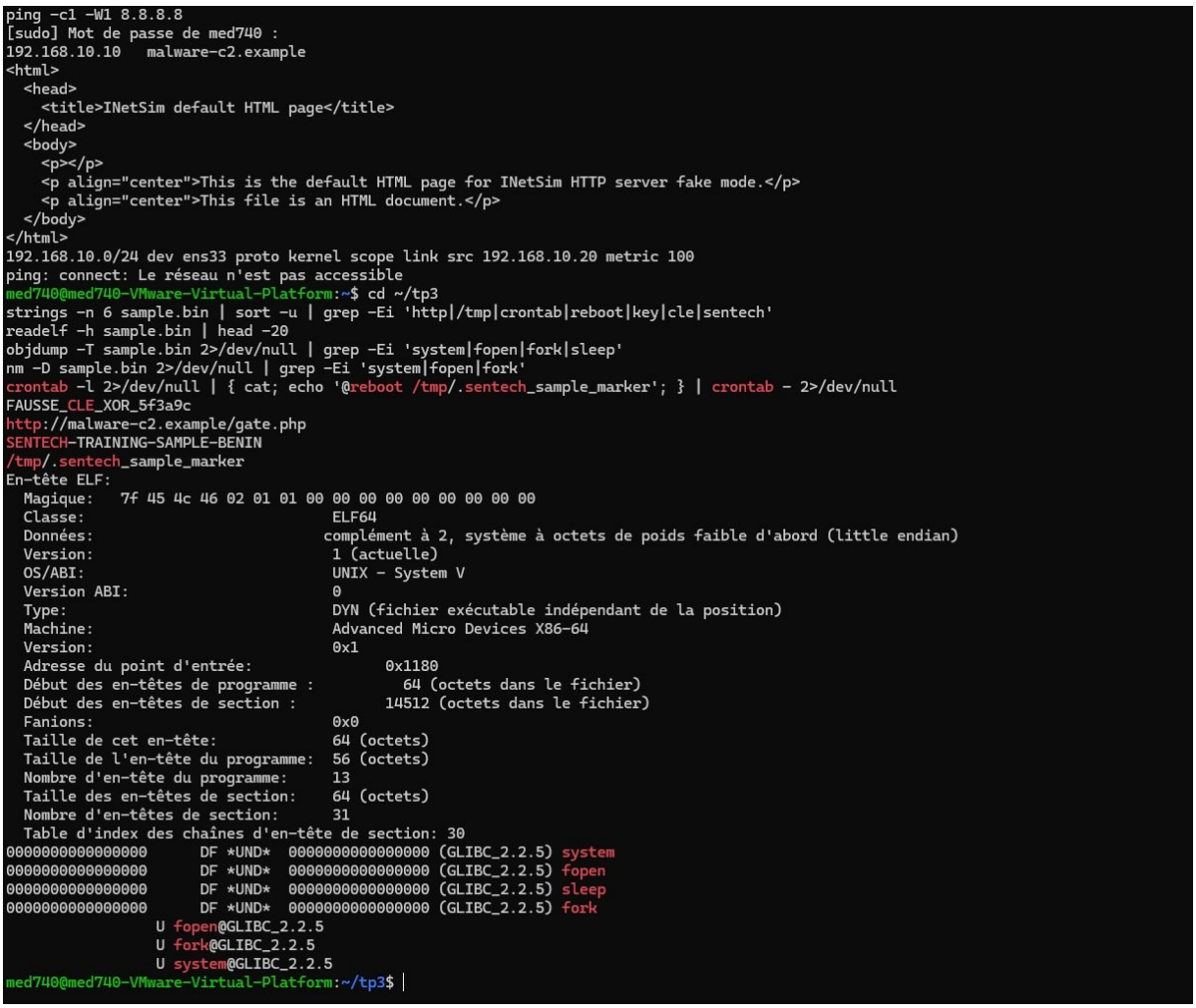


Figure 6 – Haut : preuves (a) et (b) ; bas : extraction des chaînes, en-tête ELF et symboles (Partie 2)

Écarts par rapport à l'énoncé

Deux adaptations ont été nécessaires et sont documentées pour garder l'analyse honnête :

- **DNS d'INetSim défaillant.** Le service est annoncé « started » mais aucun socket n'écoute sur le port 53 (ss vide, requête Python directe en timeout, resolvectl en « Connection refused »). Versions : INetSim 1.3.2+dfsg.1-8 et libnet-dns-perl 1.57-1 ; le démarrage affiche l'avertissement « Attempt to start Net::DNS::Nameserver in a subprocess ». Contournement : un répondeur DNS minimal en Python (/root/fakedns.py) répond à toute requête A par 192.168.10.10. Le HTTP et le HTTPS restent simulés par INetSim.
- **curl absent.** La VM n'a pas Internet et curl n'y est pas installé. Un script /usr/local/bin/curl exécute la même requête avec wget (présent). La requête réseau reste un GET HTTP vers le même domaine ; seuls l'exécutable (wget) et le User-Agent (Wget/1.21.4) diffèrent.

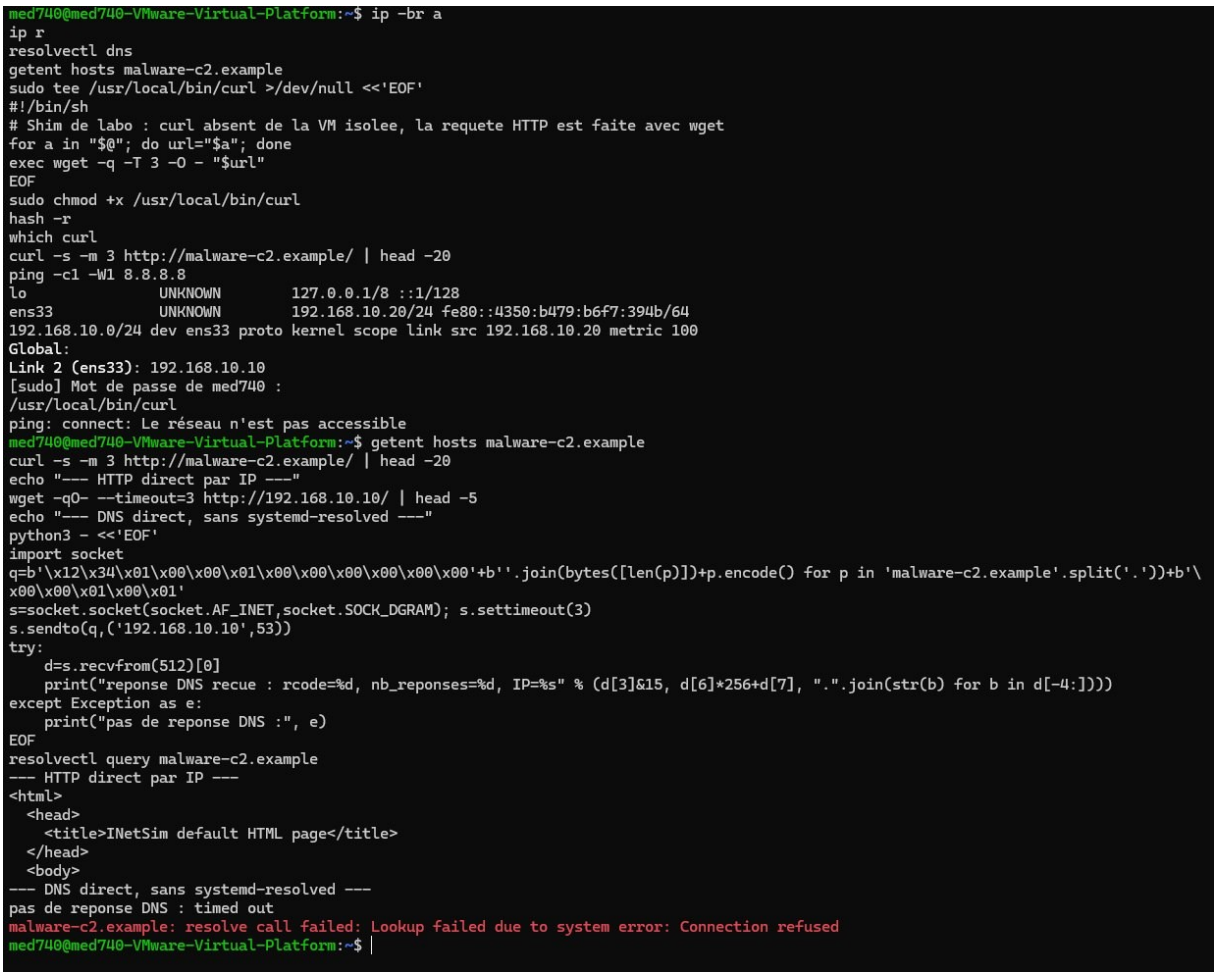


Figure 7 – Diagnostic Ubuntu : HTTP direct par IP fonctionnel, DNS en timeout et « Connection refused »

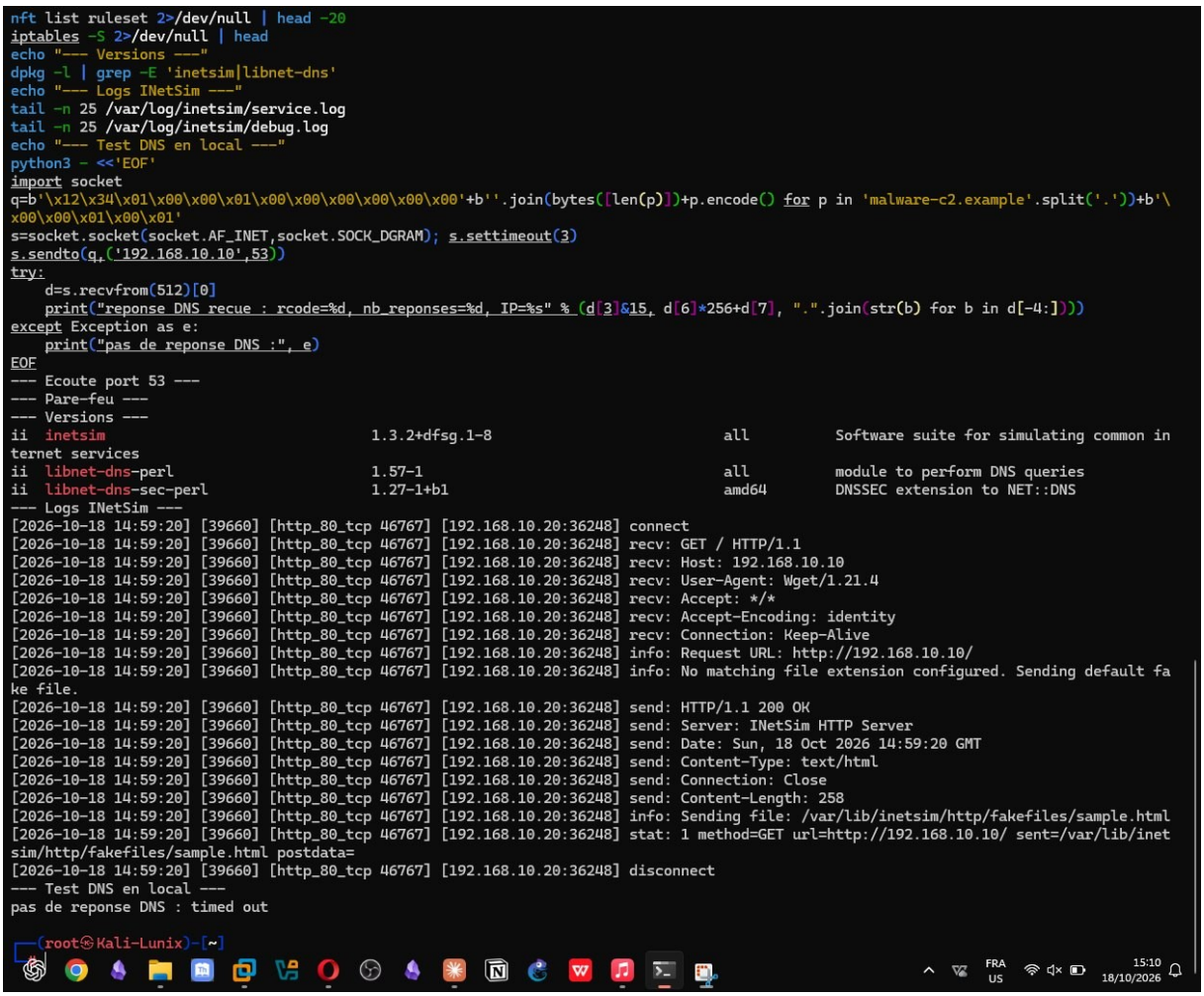


Figure 8 – Diagnostic Kali : aucun socket en écoute sur le port 53, pas de pare-feu, versions, log INetSim

```
┌──(root㉿Kali-Lunix)-[~]
└─# cat > /root/fakedns.py <<'EOF'
#!/usr/bin/env python3
# Repondeur DNS de labo : repond a toute requete A avec l'IP de la Kali.
# Equivalent du service DNS d'INetSim (qui ne demarre pas sur cette Kali).
import socket, struct
IP = "192.168.10.10"

def parse(d):
    i, labels = 12, []
    while d[i]:
        n = d[i]
        labels.append(d[i+1:i+1+n].decode(errors="ignore"))
        i += n + 1
    i += 1
    qtype = struct.unpack(">H", d[i:i+2])[0]
    return ".".join(labels), qtype, i + 4

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(("0.0.0.0", 53))
print("fakedns : ecoute sur 0.0.0.0:53, reponse =", IP, flush=True)
while True:
    d, addr = s.recvfrom(512)
    try:
        name, qtype, qend = parse(d)
    except Exception:
        continue
    question = d[12:qend]
    if qtype == 1:
        head = d[:2] + struct.pack(">HHHHH", 0x8180, 1, 1, 0, 0)
        rr = b"\xc0\x0c" + struct.pack(">HHIH", 1, 1, 60, 4) + socket.inet_aton(IP)
        s.sendto(head + question + rr, addr)
    else:
        head = d[:2] + struct.pack(">HHHHH", 0x8180, 1, 0, 0, 0)
        s.sendto(head + question, addr)
    print(addr[0], "demande", name, "type", qtype, flush=True)
EOF
python3 /root/fakedns.py
fakedns : ecoute sur 0.0.0.0:53, reponse = 192.168.10.10
192.168.10.20 demande ntp.ubuntu.com type 1
192.168.10.20 demande ntp.ubuntu.com type 28
192.168.10.20 demande malware-c2.example type 28
192.168.10.20 demande malware-c2.example type 28
192.168.10.20 demande malware-c2.example type 1
192.168.10.20 demande malware-c2.example type 28
```

Figure 9 – Répondeur DNS de remplacement (fakedns.py) en écoute sur 0.0.0.0:53

```
med740@med740-VMware-Virtual-Platform:~$ ip -br a
ip r
resolvectl dns
getent hosts malware-c2.example
sudo tee /usr/local/bin/curl >/dev/null <<'EOF'
#!/bin/sh
# Shim de labo : curl absent de la VM isolee, la requete HTTP est faite avec wget
for a in "$@"; do url="$a"; done
exec wget -q -T 3 -O - "$url"
EOF
sudo chmod +x /usr/local/bin/curl
hash -r
which curl
curl -s -m 3 http://malware-c2.example/ | head -20
ping -c1 -W1 8.8.8.8
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            UNKNOWN        192.168.10.20/24 fe80::4350:b479:b6f7:394b/64
192.168.10.0/24 dev ens33 proto kernel scope link src 192.168.10.20 metric 100
Global:
Link 2 (ens33): 192.168.10.10
[sudo] Mot de passe de med740 :
/usr/local/bin/curl
ping: connect: Le réseau n'est pas accessible
med740@med740-VMware-Virtual-Platform:~$
```

Figure 10 – Script curl basé sur wget (création, vérification du réseau isolé)

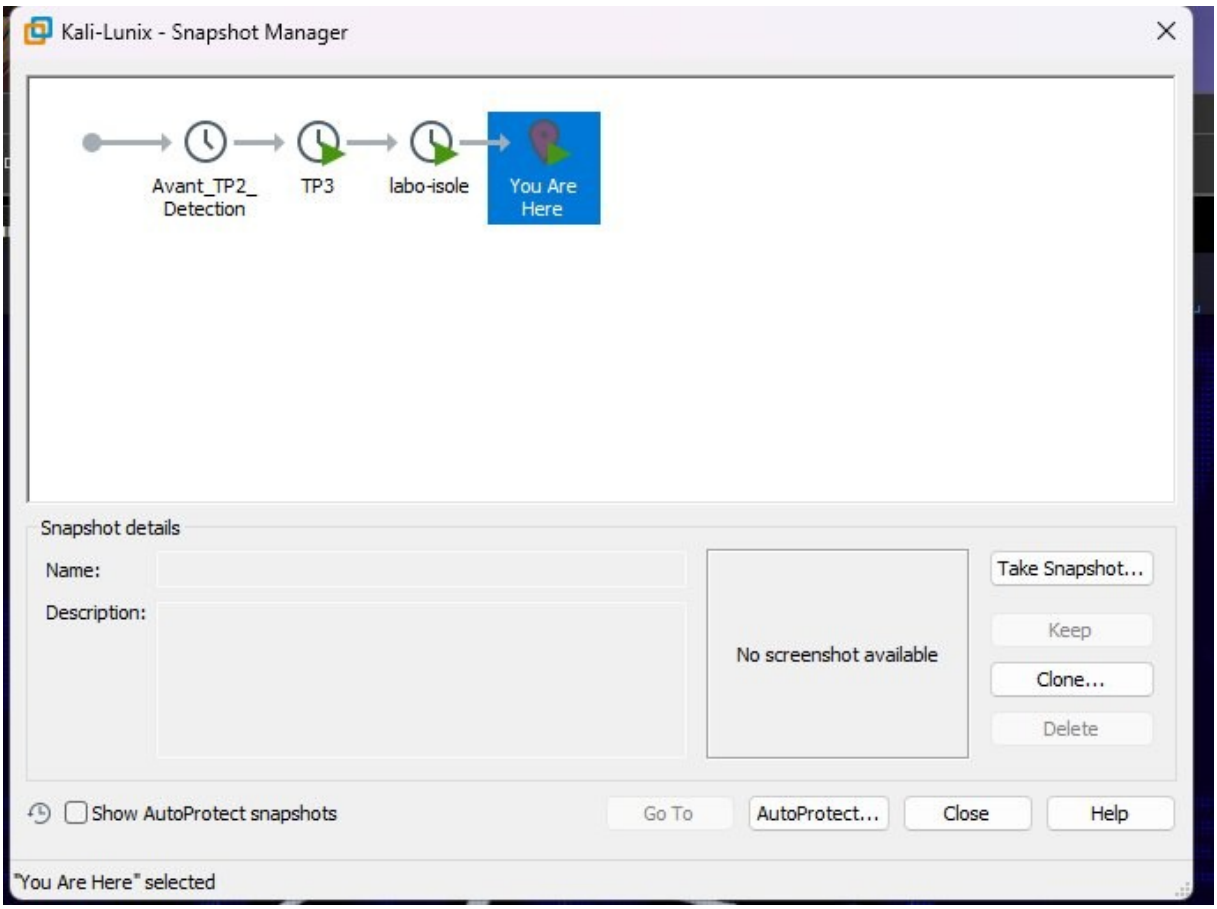


Figure 11 – Snapshot « labo-isole » de Kali-Lunix

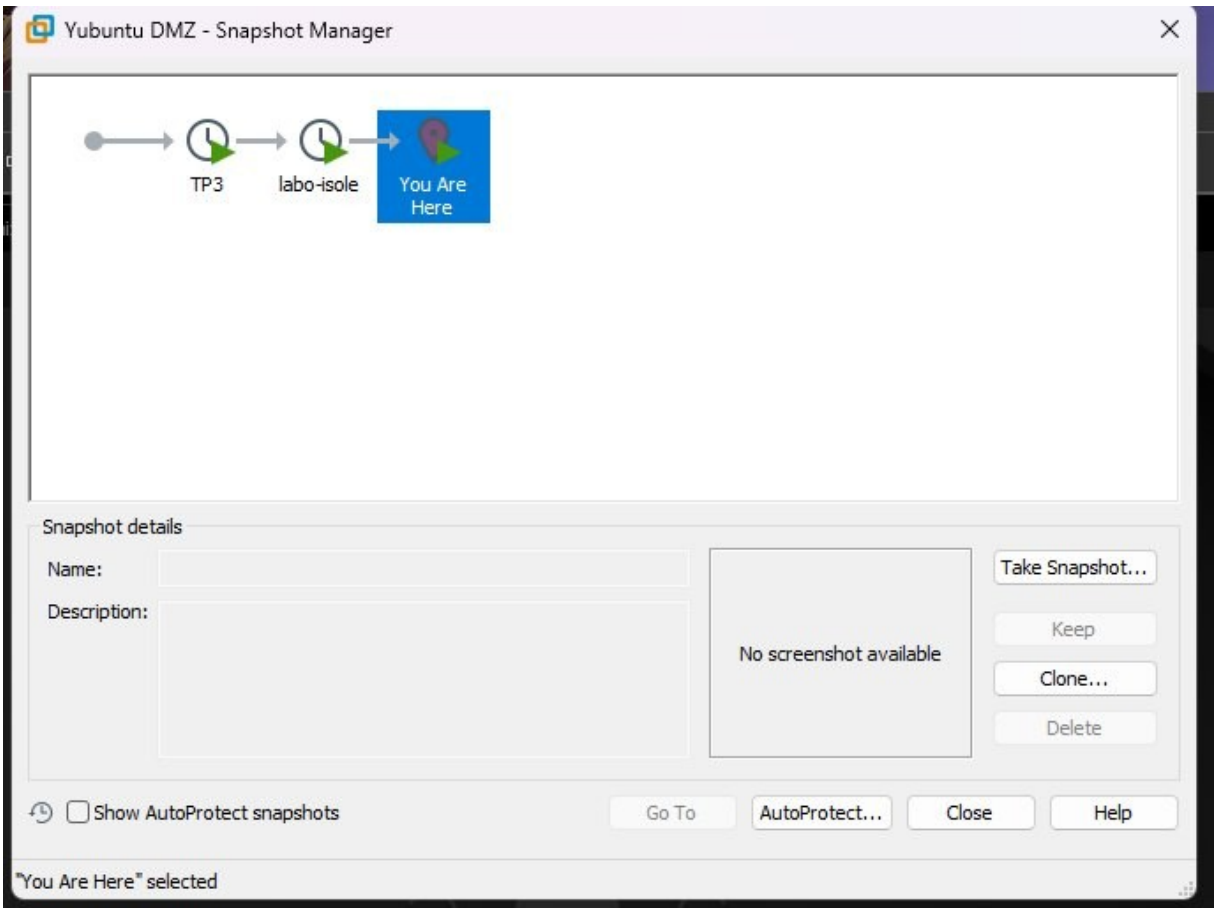


Figure 12 – Snapshot « labo-isole » de Yubuntu DMZ (état propre avant chaque exécution)

3. Partie 1 – Triage et identification

Élément	Valeur
SHA-256 (sample.bin)	58fc0097247b5a77bfc2a540b2fbc8c60df7349cf24464cce87c3e2abf353a5c
Type réel (file)	ELF 64-bit LSB pie executable, dynamically linked, not stripped
Architecture	x86-64 (GNU/Linux 3.2.0), interpréteur /lib64/ld-linux-x86-64.so.2
BuildID (sha1)	71081b873c91fb81595ee934d96e2c76b5f1df86
Taille	16 496 octets
Version empaquetée	sample_packed.bin : UPX 4.2.2, 6 936 octets (42,05 %), SHA-256 9543717941e1990ce3884ca8184a39277180ed8f40013e1ecab3b34567eba645

Pourquoi ne pas se fier à l'extension : le nom d'un fichier est choisi par son auteur et n'engage à rien. Un exécutable peut s'appeler facture.pdf. La commande file lit les octets d'en-tête (ici la signature \x7fELF) et donne le type réel, indépendamment du nom.

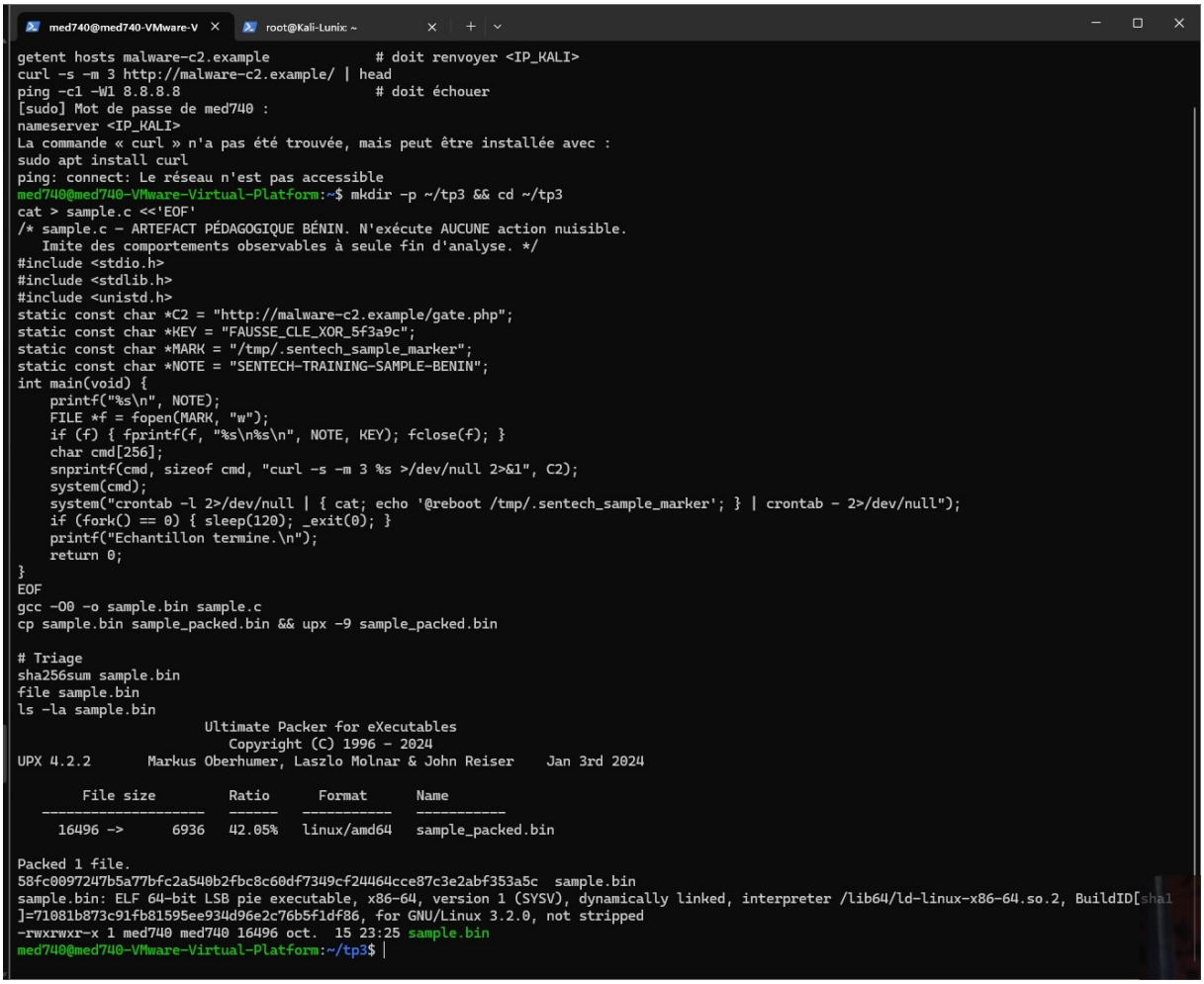


Figure 13 – Compilation, empaquetage UPX puis triage (sha256sum, file, ls -la) – l'échantillon n'a pas été exécuté

4. Partie 2 – Analyse statique (sans exécution)

4.1 Chaînes de caractères

Indicateur	Interprétation	Technique ATT&CK présumée
http://malware-c2.example/gate.php	URL d'un serveur de commande et contrôle ; gate.php est typique d'un panneau C2	T1071.001
curl -s -m 3 %s >/dev/null 2>&1	Requête HTTP silencieuse (timeout 3 s, sortie masquée) : une balise	T1059.004
crontab -l … echo '@reboot /tmp/.sentech_sample_marker' … | crontab -	Ajout d'une entrée cron exécutée au redémarrage : persistance	T1053.003
/tmp/.sentech_sample_marker	Fichier déposé, caché par le point initial	T1564.001
FAUSSE_CLE_XOR_5f3a9c	Clé codée en dur : laisse présager chiffrement ou obfuscation (ici un leurre)	T1573.001 / T1027
SENTECH-TRAINING-SAMPLE-BENIN	Message affiché et écrit dans le marqueur ; identifiant utile à la détection	aucune

4.2 En-tête ELF et symboles importés

readelf -h : ELF64, little endian, type DYN (exécutable indépendant de la position), machine x86-64, point d'entrée 0x1180, 13 en-têtes de programme, 31 en-têtes de section. objdump -T et nm -D concordent avec radare2 (ii).

Symbole	Ce qu'il laisse présager
system	Exécution de commandes shell arbitraires : ici il lance curl et crontab
fopen (avec fprintf, fclose)	Écriture d'un fichier sur le disque : dépôt d'un artefact
fork (avec sleep)	Création d'un processus enfant qui reste en veille : processus résident

4.3 Empaquetage

Critère	sample.bin	sample_packed.bin
Entropie	2,06 bits/octet	7,28 bits/octet (contenu compressé)
Signature du packer	aucune	upx -t : [OK] ; « UPX! » et bannière UPX 4.22 dans le binaire
Chaînes significatives	toutes lisibles	absentes (seul le fragment « http://malware-c2.examp » reste visible)
Nombre de chaînes brut	100	133 (non discriminant : les octets compressés créent des chaînes parasites)
SHA-256	58fc0097…353a5c	9543717941e1…eba645

Verdict : sample_packed.bin est empaqueté avec UPX 4.2.2. Preuve de dépaquetage : upx -d produit sample_unpacked.bin dont le SHA-256 est strictement identique à celui de sample.bin (58fc0097…353a5c) et les chaînes réapparaissent. Le hachage de la version empaquetée est différent : le hash est donc un IOC fragile.

```
med740@med740-VMware-Virtual-Platform:~/tp3$ cd ~/tp3
echo "Original : $(strings sample.bin | wc -l) chaines"
echo "Empaquete : $(strings sample_packed.bin | wc -l) chaines"
upx -t sample_packed.bin
python3 - <<'EOF'
import math, collections
for p in ("sample.bin","sample_packed.bin"):
    d=open(p,"rb").read(); c=collections.Counter(d)
    H=-sum(n/len(d)*math.log2(n/len(d)) for n in c.values())
    print(f"{p}: entropie={H:.2f} bits/octet")
EOF
upx -d -o sample_unpacked.bin sample_packed.bin
strings sample_unpacked.bin | grep -i malware-c2
Original : 100 chaines
Empaquete : 133 chaines
                       Ultimate Packer for eXecutables
                          Copyright (C) 1996 - 2024
UPX 4.2.2       Markus Oberhumer, Laszlo Molnar & John Reiser    Jan 3rd 2024

testing sample_packed.bin [OK]

Tested 1 file.
sample.bin: entropie=2.06 bits/octet
sample_packed.bin: entropie=7.28 bits/octet
                       Ultimate Packer for eXecutables
                          Copyright (C) 1996 - 2024
UPX 4.2.2       Markus Oberhumer, Laszlo Molnar & John Reiser    Jan 3rd 2024

        File size         Ratio      Format      Name
   --------------------   ------   -----------   -----------
     24479 <-      6936   28.33%   linux/amd64   sample_unpacked.bin

Unpacked 1 file.
http://malware-c2.example/gate.php
med740@med740-VMware-Virtual-Platform:~/tp3$
```

Figure 14 – Comparaison des chaînes, upx -t, entropie, dépaquetage

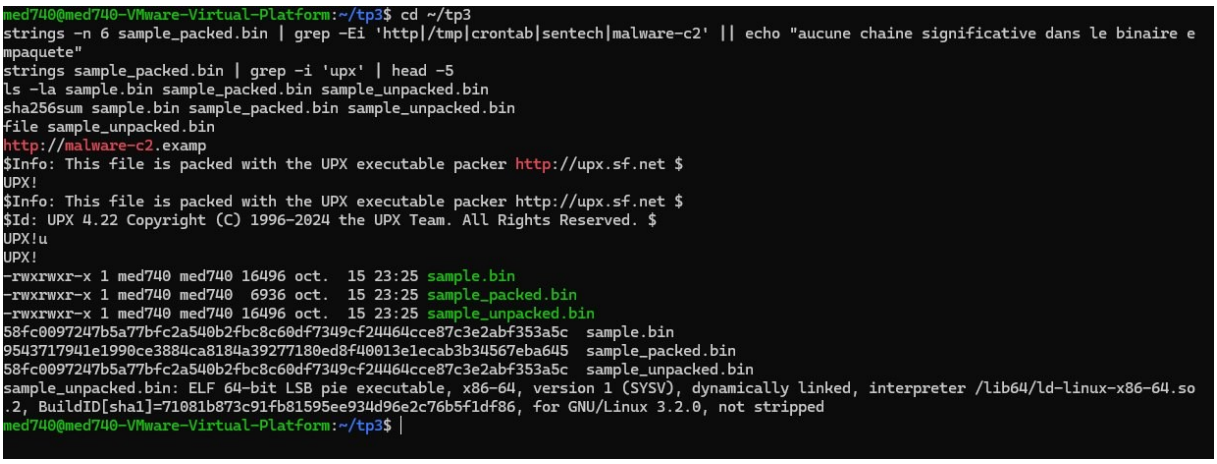


Figure 15 – Chaînes résiduelles de la version empaquetée, tailles et SHA-256 identiques après dépaquetage

4.4 Désassemblage de main (radare2)

Ordre des appels dans main, lu sans exécuter : puts(NOTE) → fopen(MARK, "w") → fprintf("%s\n%s\n", NOTE, KEY) → fclose → snprintf(« curl -s -m 3 %s … », C2) → system(curl) → system(crontab …) → fork. Le fils (retour 0) appelle sleep(0x78) soit 120 s puis _exit(0) ; le père affiche « Echantillon termine. » et retourne 0.

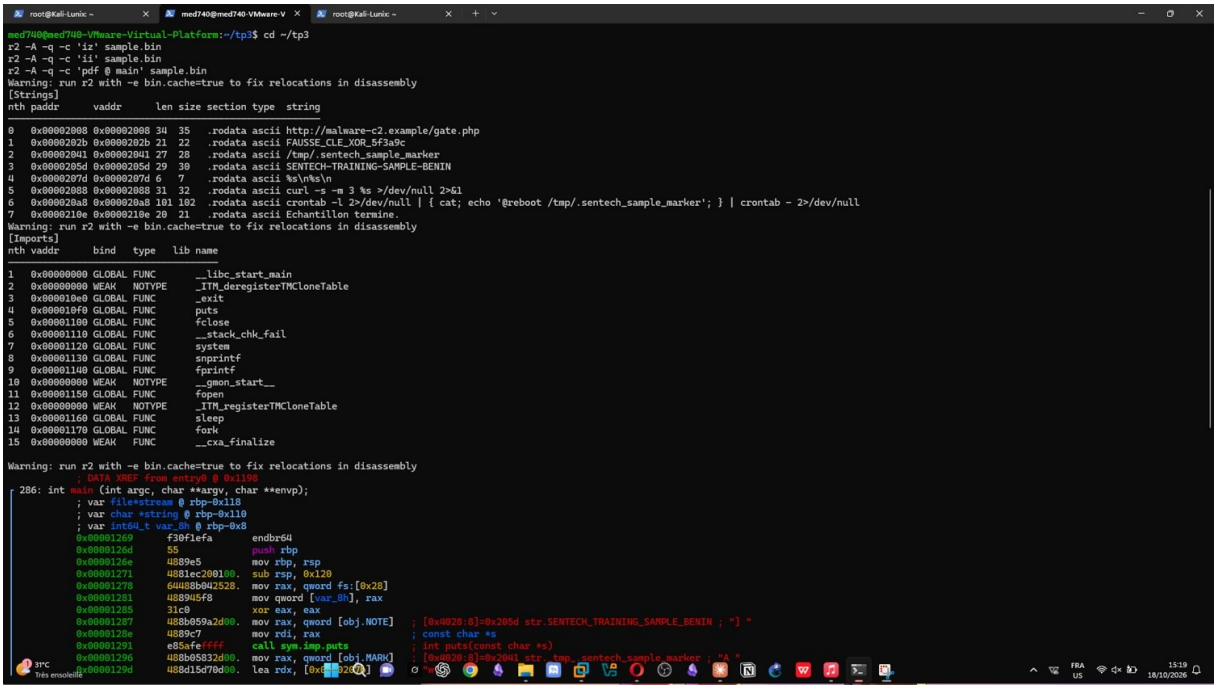


Figure 16 – radare2 : iz (chaînes), ii (imports) et début de main

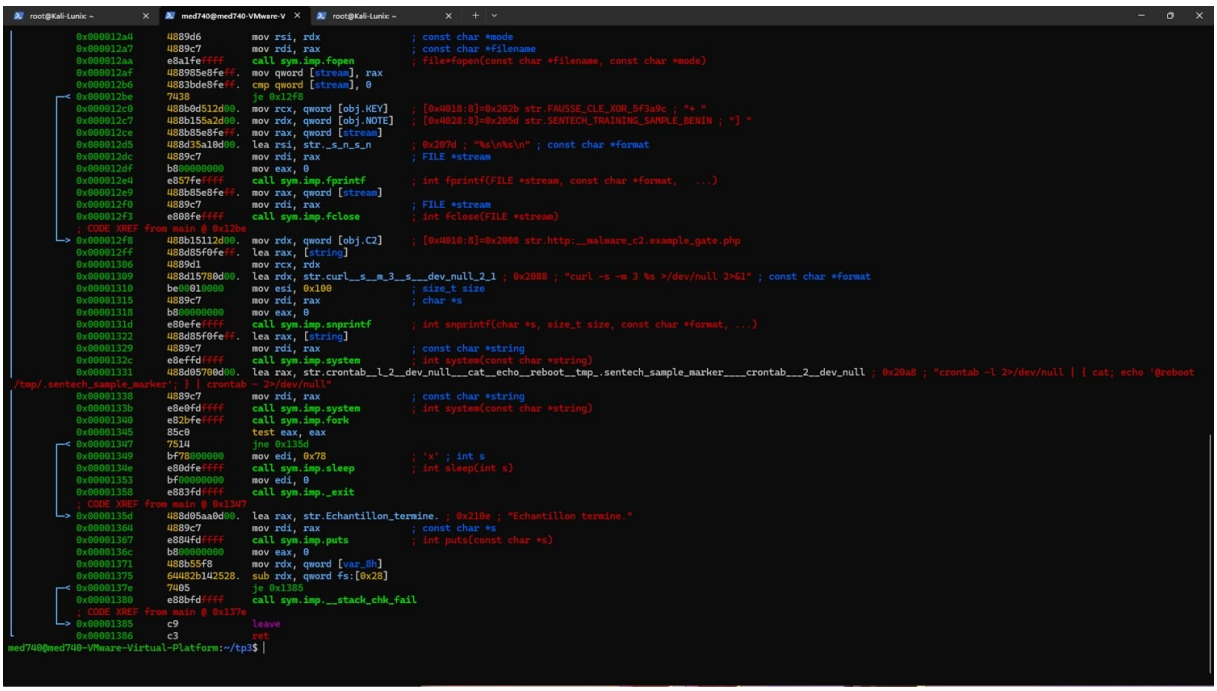


Figure 17 – radare2 : suite de main (fopen, system, fork, sleep)

5. Partie 3 – Analyse dynamique (exécution contrôlée)

Protocole : retour au snapshot « labo-isole », INetSim et fakedns actifs sur la Kali, puis surveillance parallèle sur l'Ubuntu : inotifywait (fichiers), tcpdump sur ens33 ports 53 et 80 (réseau), pstree/ps (processus), crontab avant/après (persistance) et strace -f -e trace=file,process,network (appels système). L'échantillon est lancé par l'utilisateur med740.

5.1 Observations

Observation	Outil	Preuve	Technique
Fichier marqueur caché créé dans /tmp	inotifywait, strace	CREATE/MODIFY/ CLOSE_WRITE .sentech_sample_marker ; openat(..., O_WRONLY|O_CREAT|O_TRUNC) ; contenu : note + FAUSSE_CLE_XOR_5f3a9c	T1564.001
Balise HTTP vers le domaine C2	tcpdump, strace, log INetSim	15:43:55 : requête A malware-c2.example → 192.168.10.10 ; SYN vers :80 ; GET /gate.php ; 200 OK ; connect(...192.168.10.10:80) ; log INetSim (User-Agent Wget/1.21.4)	T1071.001
Entrée @reboot ajoutée au crontab	diff crontab, inotifywait, strace	diff : > @reboot /tmp/.sentech_sample_marker ; création de crontabs/tmp.za3igY renommé en crontabs/med740	T1053.003
Processus enfant résident	pstree, ps	sample.bin PID 4136, parent 1 (père terminé, enfant orphelin en veille 120 s) ; commandes lancées via sh -c	T1106, T1059.004

Chaîne d'exécution vue par strace : sample.bin → sh -c « curl ... » → /usr/local/bin/curl (script de labo) → wget → connect(192.168.10.10:80), puis sh -c « crontab -l ... \| crontab - » et un clone pour le processus résident.

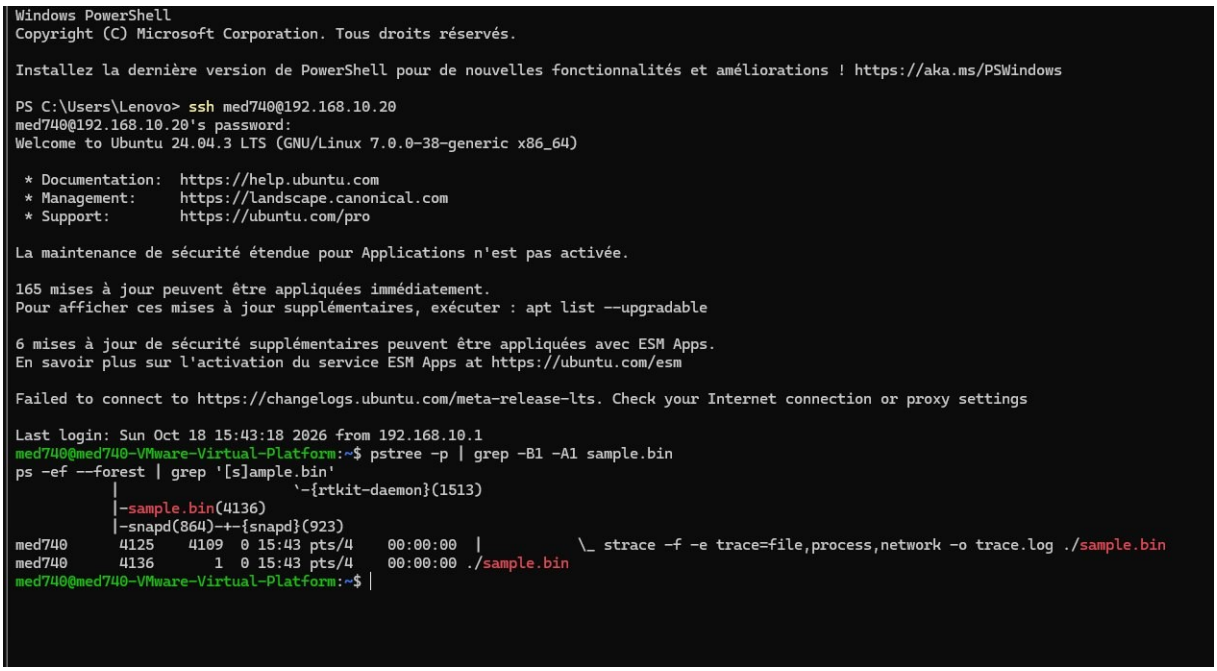


Figure 18 – pstree et ps : processus sample.bin résident, parent 1 (PID 4136)

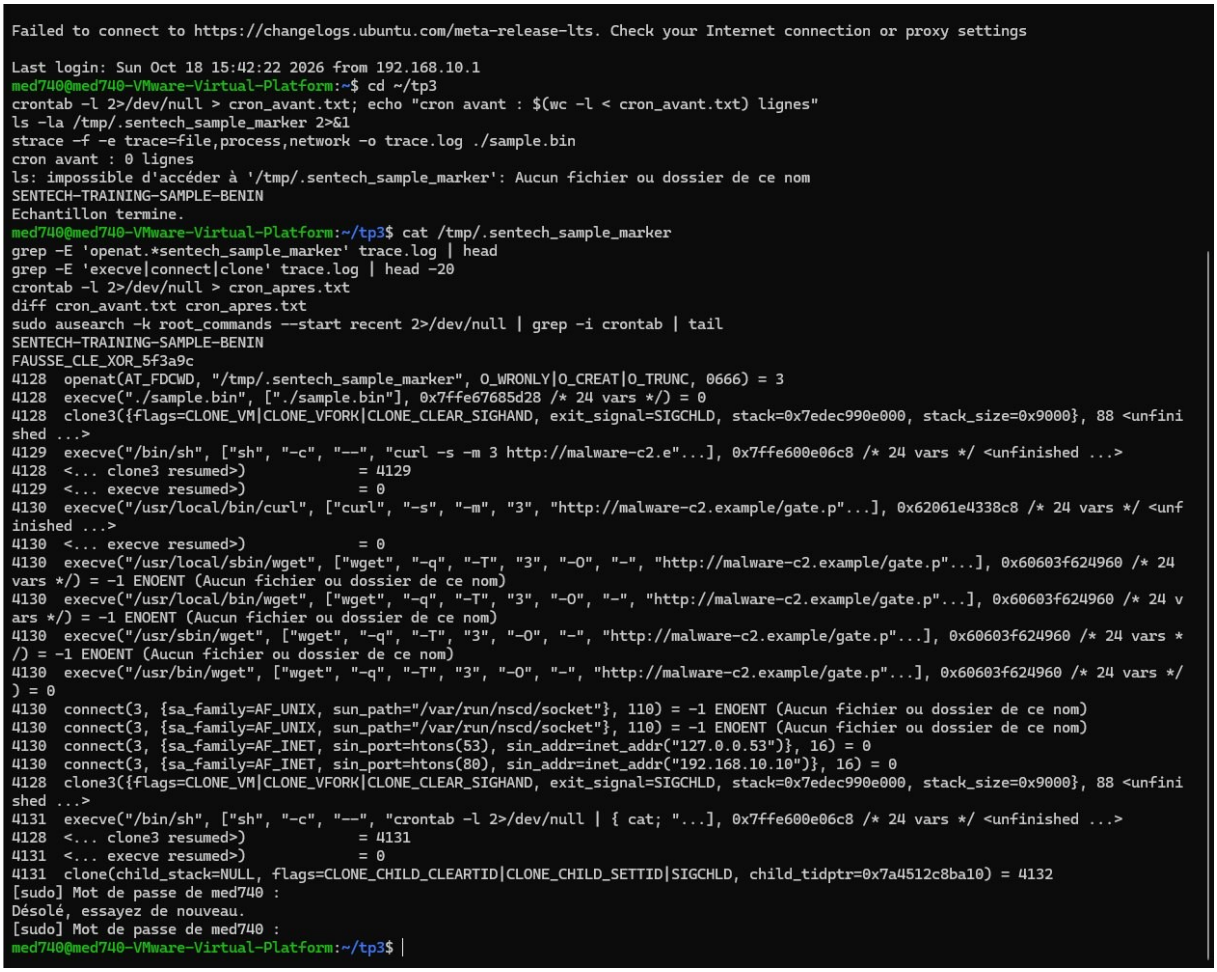


Figure 19 – Première exécution : état avant, sortie du programme, marqueur, strace, diff du crontab vide

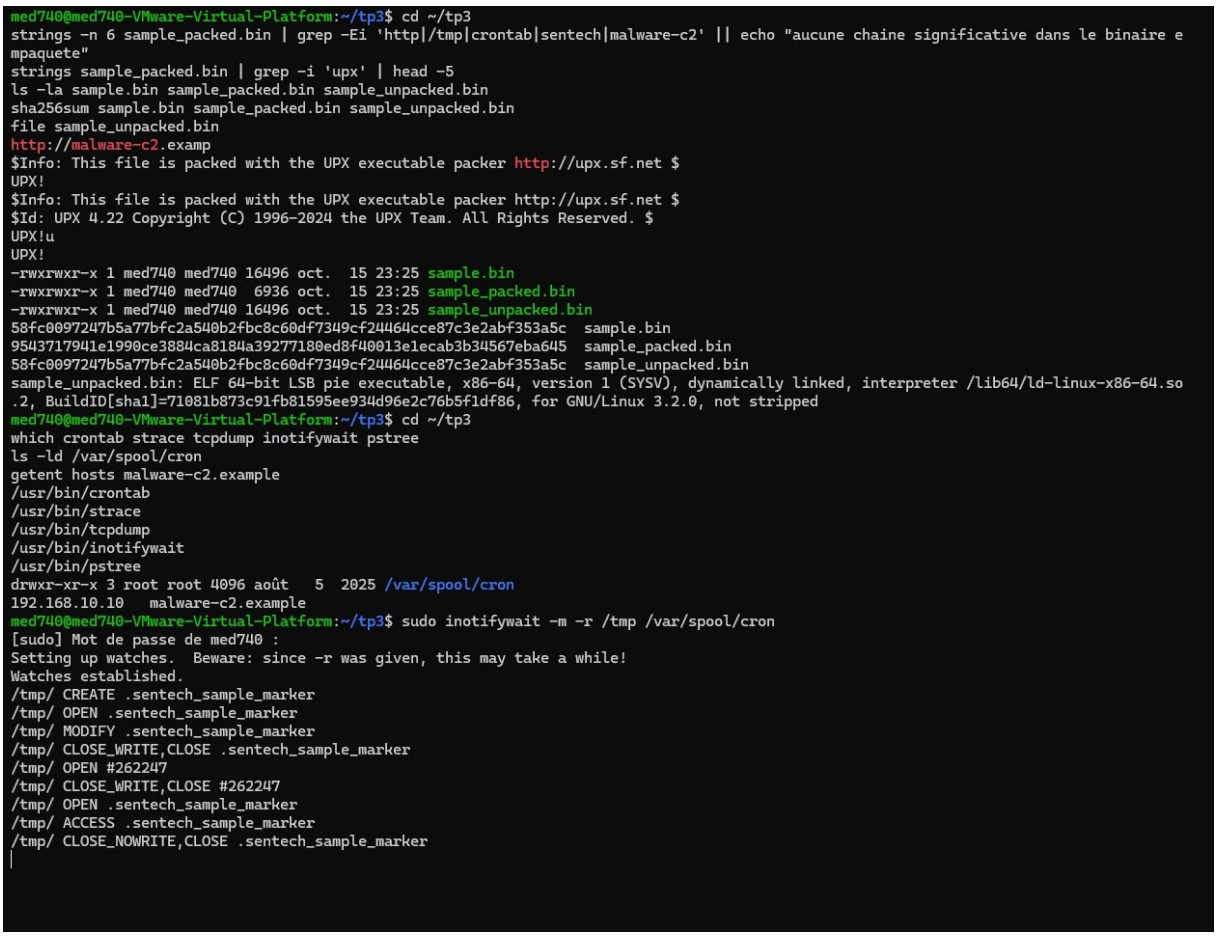


Figure 20 – Première exécution : événements inotifywait dans /tmp (aucun événement dans /var/spool/cron)

```
6 mises à jour de sécurité supplémentaires peuvent être appliquées avec ESM Apps.
En savoir plus sur l'activation du service ESM Apps at https://ubuntu.com/esm

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Sun Oct 18 14:44:59 2026 from 192.168.10.1
med740@med740-VMware-Virtual-Platform:~$ sudo tcpdump -i ens33 -n -l 'port 53 or port 80'
[sudo] Mot de passe de med740 :
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes
15:43:16.489277 IP 192.168.10.20.47842 > 192.168.10.10.53: 46275+ AAAA? connectivity-check.ubuntu.com. (47)
15:43:16.490267 IP 192.168.10.10.53 > 192.168.10.20.47842: 46275 0/0/0 (47)
15:43:16.492462 IP 192.168.10.20.48522 > 192.168.10.10.53: 23169+ A? connectivity-check.ubuntu.com. (47)
15:43:16.492758 IP 192.168.10.20.55972 > 192.168.10.10.53: 7998+ AAAA? connectivity-check.ubuntu.com. (47)
15:43:16.493626 IP 192.168.10.10.53 > 192.168.10.20.48522: 23169 1/0/0 A 192.168.10.10 (63)
15:43:16.493896 IP 192.168.10.10.53 > 192.168.10.20.55972: 7998 0/0/0 (47)
15:43:55.367718 IP 192.168.10.20.42636 > 192.168.10.10.53: 25946+ A? malware-c2.example. (36)
15:43:55.368010 IP 192.168.10.20.59197 > 192.168.10.10.53: 32898+ AAAA? malware-c2.example. (36)
15:43:55.369014 IP 192.168.10.10.53 > 192.168.10.20.42636: 25946 1/0/0 A 192.168.10.10 (52)
15:43:55.369440 IP 192.168.10.10.53 > 192.168.10.20.59197: 32898 0/0/0 (36)
15:43:55.378111 IP 192.168.10.20.58212 > 192.168.10.10.80: Flags [S], seq 3114496488, win 64240, options [mss 1460,sackOK,TS val 206311
7297 ecr 0,nop,wscale 10], length 0
15:43:55.379294 IP 192.168.10.10.80 > 192.168.10.20.58212: Flags [S.], seq 4204143096, ack 3114496489, win 65160, options [mss 1460,sac
kOK,TS val 2847505368 ecr 2063117297,nop,wscale 9], length 0
15:43:55.379344 IP 192.168.10.20.58212 > 192.168.10.10.80: Flags [.], ack 1, win 63, options [nop,nop,TS val 2063117298 ecr 2847505368]
, length 0
15:43:55.381638 IP 192.168.10.20.58212 > 192.168.10.10.80: Flags [P.], seq 1:142, ack 1, win 63, options [nop,nop,TS val 2063117301 ecr
 2847505368], length 141: HTTP: GET /gate.php HTTP/1.1
15:43:55.382623 IP 192.168.10.10.80 > 192.168.10.20.58212: Flags [.], ack 142, win 127, options [nop,nop,TS val 2847505371 ecr 20631173
01], length 0
15:43:55.425799 IP 192.168.10.10.80 > 192.168.10.20.58212: Flags [P.], seq 1:151, ack 142, win 127, options [nop,nop,TS val 2847505414
ecr 2063117301], length 150: HTTP: HTTP/1.1 200 OK
15:43:55.425841 IP 192.168.10.20.58212 > 192.168.10.10.80: Flags [.], ack 151, win 63, options [nop,nop,TS val 2063117345 ecr 284750541
4], length 0
15:43:55.426673 IP 192.168.10.10.80 > 192.168.10.20.58212: Flags [P.], seq 151:409, ack 142, win 127, options [nop,nop,TS val 284750541
5 ecr 2063117345], length 258: HTTP
15:43:55.426688 IP 192.168.10.20.58212 > 192.168.10.10.80: Flags [.], ack 409, win 63, options [nop,nop,TS val 2063117346 ecr 284750541
5], length 0
15:43:55.432734 IP 192.168.10.10.80 > 192.168.10.20.58212: Flags [F.], seq 409, ack 142, win 127, options [nop,nop,TS val 2847505421 ec
r 2063117346], length 0
15:43:55.437808 IP 192.168.10.20.58212 > 192.168.10.10.80: Flags [F.], seq 142, ack 410, win 63, options [nop,nop,TS val 2063117357 ecr
 2847505421], length 0
15:43:55.438503 IP 192.168.10.10.80 > 192.168.10.20.58212: Flags [.], ack 143, win 127, options [nop,nop,TS val 2847505427 ecr 20631173
57], length 0
15:48:17.235278 IP 192.168.10.20.54754 > 192.168.10.10.53: 53671+ AAAA? connectivity-check.ubuntu.com. (47)
15:48:17.236518 IP 192.168.10.10.53 > 192.168.10.20.54754: 53671 0/0/0 (47)
15:48:17.239155 IP 192.168.10.20.58439 > 192.168.10.10.53: 51155+ A? connectivity-check.ubuntu.com. (47)
15:48:17.239640 IP 192.168.10.20.43910 > 192.168.10.10.53: 55303+ AAAA? connectivity-check.ubuntu.com. (47)
15:48:17.240440 IP 192.168.10.10.53 > 192.168.10.20.58439: 51155 1/0/0 A 192.168.10.10 (63)
15:48:17.240971 IP 192.168.10.10.53 > 192.168.10.20.43910: 55303 0/0/0 (47)
```

Figure 21 – Extrait tcpdump de la balise C2 : DNS (A et AAAA) puis GET /gate.php, 200 OK

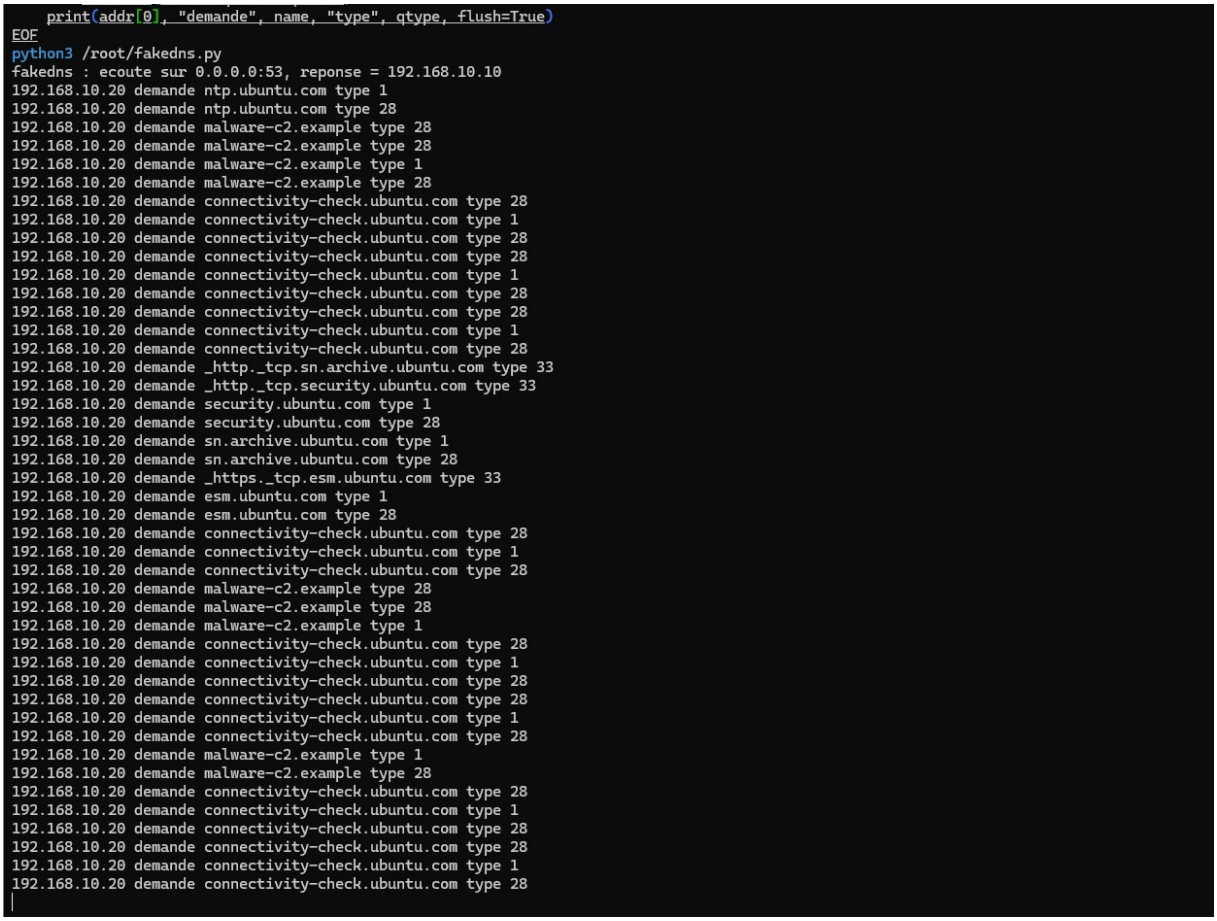


Figure 22 – Côté Kali : requêtes DNS reçues par le répondeur (malware-c2.example type 1 et 28)

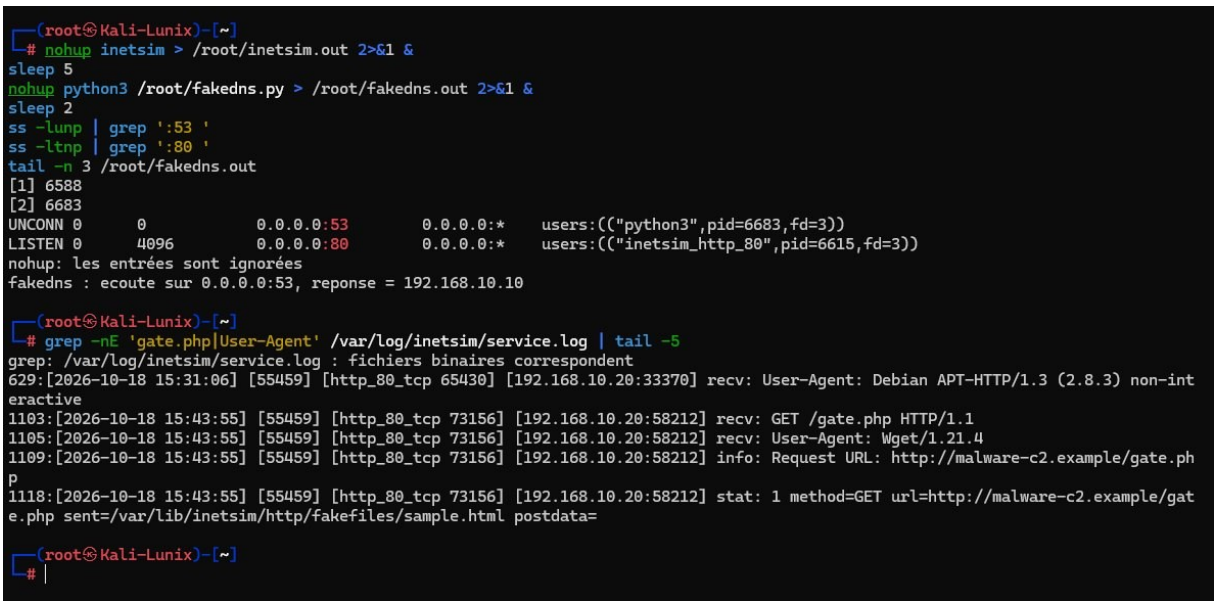


Figure 23 – Log INetSim : GET /gate.php, User-Agent Wget/1.21.4, session du 18/10 à 15:43:55

5.2 Limite de l'observation : la persistance masquée par strace

Lors de la première exécution, le diff du crontab était vide alors que le programme lançait bien la commande crontab. Cause établie à partir de la trace : /usr/bin/crontab est setgid (S_ISGID|0755) et écrit dans /var/spool/cron/crontabs (0730, groupe crontab) ; or strace lancé par un utilisateur ordinaire fait ignorer le bit setgid. Résultat : openat(crontabs/tmp.FxUUmL, O_CREAT|O_EXCL) = -1 EACCES, et même la lecture de

crontabs/med740 est refusée ; l'erreur était redirigée vers /dev/null par le programme. L'outil d'observation a donc empêché un comportement de l'échantillon.

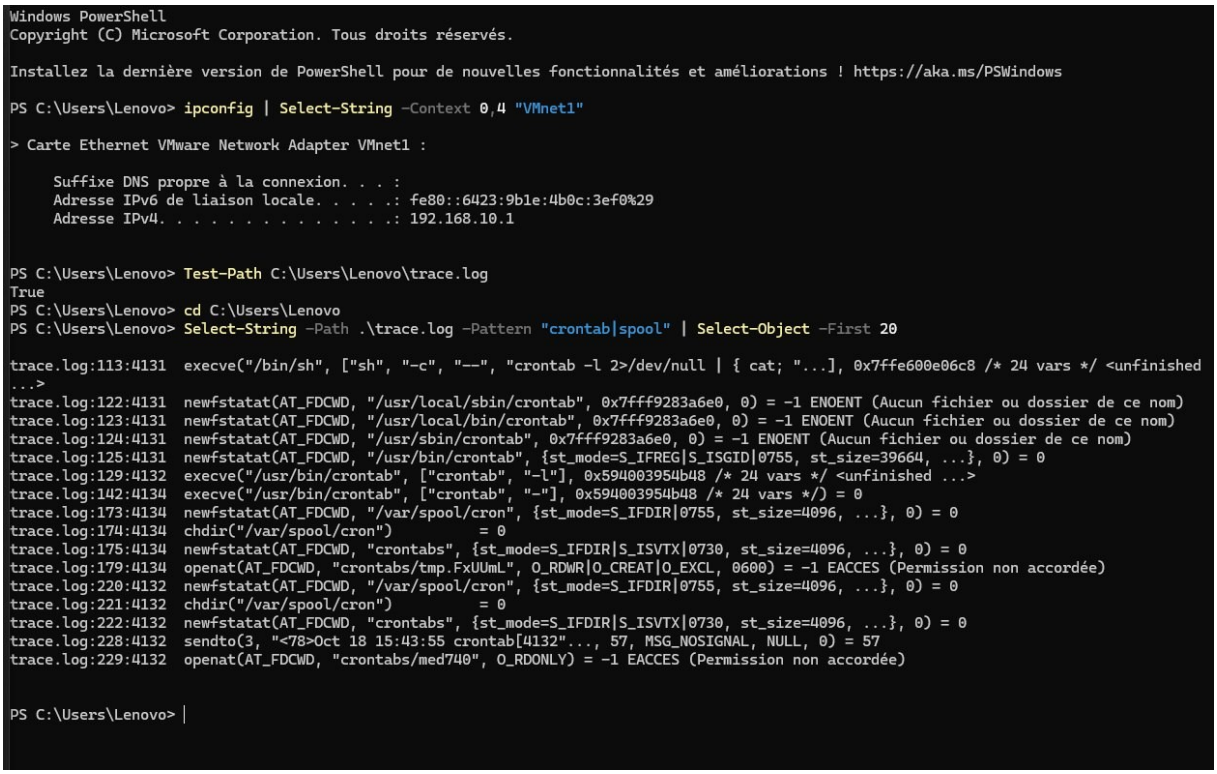


Figure 24 – Trace de la 1re exécution : crontab setgid mais openat(crontabs/tmp.) en EACCES sous strace*

5.3 Deuxième exécution : persistance observée

Après retour au snapshot propre, l'échantillon est relancé avec sudo strace -u med740 -f … : strace tourne en root et exécute l'échantillon sous le compte med740, ce qui rétablit le bit setgid de crontab. Résultats : cron avant = 0 ligne, diff = « > @reboot /tmp/.sentech_sample_marker », crontab -l affiche cette ligne, et trace2.log montre openat(crontabs/tmp.za3igY, O_CREAT|O_EXCL) = 3 puis rename(… → crontabs/med740) = 0.

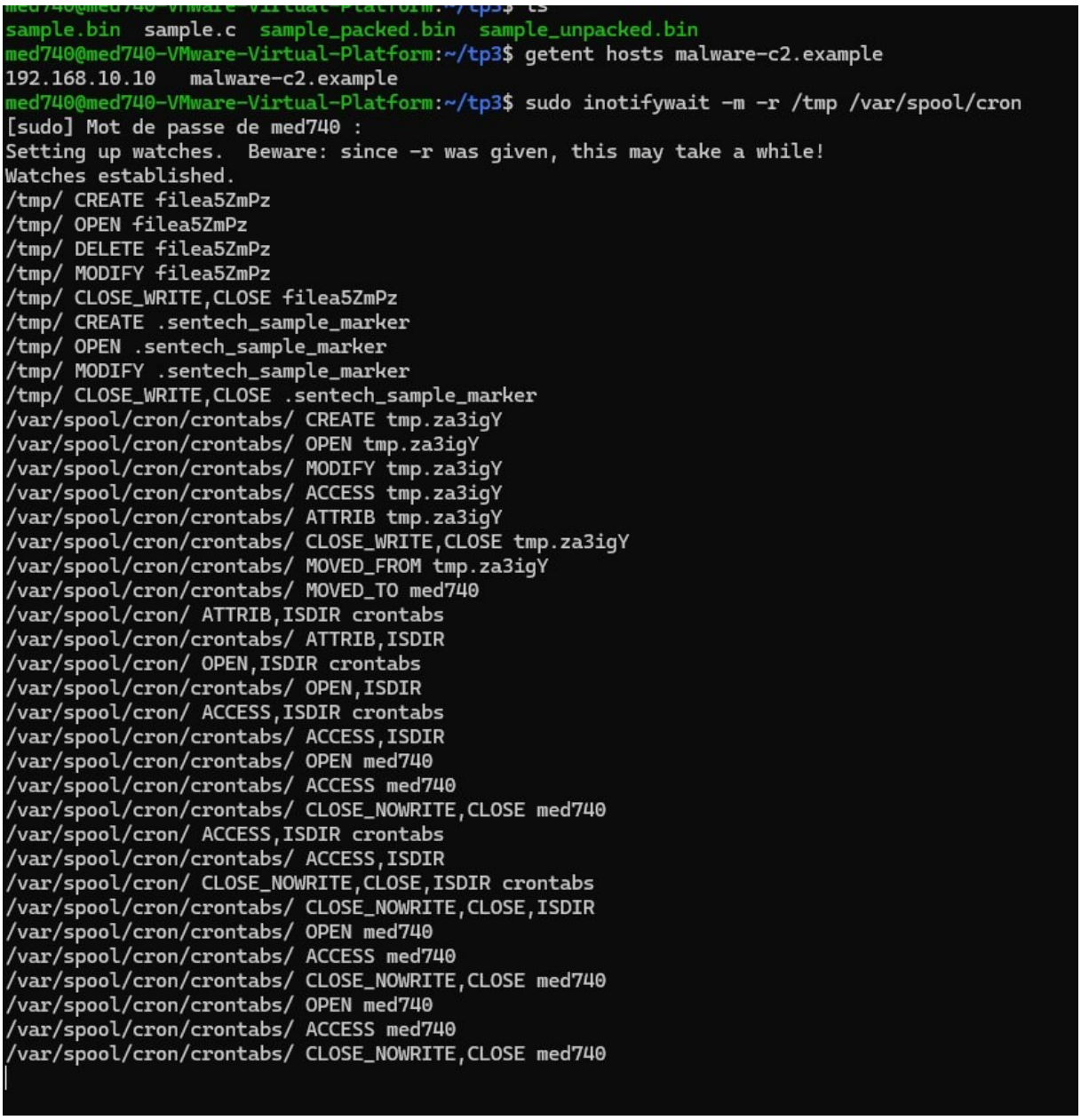


Figure 25 – 2e exécution : création de crontabs/tmp.za3igY puis MOVED_TO med740 dans /var/spool/cron/crontabs

```
med740@med740-VMware-Virtual-Platform:~$ cd ~/tp3
crontab -l 2>/dev/null > cron_avant.txt; echo "cron avant : $(wc -l < cron_avant.txt) lignes"
ls -la /tmp/.sentech_sample_marker 2>&1
sudo strace -u med740 -f -e trace=file,process,network -o trace2.log ./sample.bin
crontab -l 2>/dev/null > cron_apres.txt
diff cron_avant.txt cron_apres.txt
crontab -l
grep -nE 'crontab|spool' trace2.log | head -20
cron avant : 0 lignes
ls: impossible d'accéder à '/tmp/.sentech_sample_marker': Aucun fichier ou dossier de ce nom
[sudo] Mot de passe de med740 :
Désolé, essayez de nouveau.
[sudo] Mot de passe de med740 :
Désolé, essayez de nouveau.
[sudo] Mot de passe de med740 :
SENTECH-TRAINING-SAMPLE-BENIN
Echantillon termine.
0a1
> @reboot /tmp/.sentech_sample_marker
@reboot /tmp/.sentech_sample_marker
112:4566  execve("/bin/sh", ["sh", "-c", "--", "crontab -l 2>/dev/null | { cat; "...], 0x7ffcbd613798 /* 13 vars */ <unfinished .
..>
120:4566  newfstatat(AT_FDCWD, "/usr/local/sbin/crontab", 0x7fff9c0125e0, 0) = -1 ENOENT (Aucun fichier ou dossier de ce nom)
121:4566  newfstatat(AT_FDCWD, "/usr/local/bin/crontab", 0x7fff9c0125e0, 0) = -1 ENOENT (Aucun fichier ou dossier de ce nom)
122:4566  newfstatat(AT_FDCWD, "/usr/sbin/crontab", 0x7fff9c0125e0, 0) = -1 ENOENT (Aucun fichier ou dossier de ce nom)
123:4566  newfstatat(AT_FDCWD, "/usr/bin/crontab", {st_mode=S_IFREG|S_ISGID|0755, st_size=39664, ...}, 0) = 0
129:4567  execve("/usr/bin/crontab", ["crontab", "-l"], 0x589724541938 /* 14 vars */ <unfinished ...>
148:4569  execve("/usr/bin/crontab", ["crontab", "-"], 0x589724541938 /* 14 vars */ <unfinished ...>
168:4567  newfstatat(AT_FDCWD, "/var/spool/cron", {st_mode=S_IFDIR|0755, st_size=4096, ...}, 0) = 0
169:4567  chdir("/var/spool/cron")          = 0
170:4567  newfstatat(AT_FDCWD, "crontabs",  <unfinished ...>
185:4567  sendto(3, "<78>Oct 31 19:32:38 crontab[4567"..., 57, MSG_NOSIGNAL, NULL, 0 <unfinished ...>
188:4567  openat(AT_FDCWD, "crontabs/med740", O_RDONLY) = -1 ENOENT (Aucun fichier ou dossier de ce nom)
206:4569  newfstatat(AT_FDCWD, "/var/spool/cron",  <unfinished ...>
211:4569  chdir("/var/spool/cron")          = 0
212:4569  newfstatat(AT_FDCWD, "crontabs",  <unfinished ...>
217:4569  openat(AT_FDCWD, "crontabs/tmp.za3igY", O_RDWR|O_CREAT|O_EXCL, 0600) = 3
219:4569  rename("crontabs/tmp.za3igY", "crontabs/med740") = 0
222:4569  sendto(3, "<78>Oct 31 19:32:38 crontab[4569"..., 60, MSG_NOSIGNAL, NULL, 0) = 60
223:4569  utimensat(AT_FDCWD, "crontabs", NULL, 0) = 0
med740@med740-VMware-Virtual-Platform:~/tp3$
```

Figure 26 – 2e exécution : diff du crontab, crontab -l et extrait de trace2.log

```
PS C:\Users\Lenovo> cd C:\Users\Lenovo
PS C:\Users\Lenovo> scp med740@192.168.10.20:tp3/trace2.log .
med740@192.168.10.20's password:
trace2.log                                                                    100%   20KB   2.5MB/s   00:00
PS C:\Users\Lenovo> cat trace2.log
4563  execve("./sample.bin", ["./sample.bin"], 0x7ffd5f62dff8 /* 13 vars */) = 0
4563  access("/etc/ld.so.preload", R_OK) = -1 ENOENT (Aucun fichier ou dossier de ce nom)
4563  openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
4563  openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
4563  openat(AT_FDCWD, "/tmp/.sentech_sample_marker", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
4563  clone3({flags=CLONE_VM|CLONE_VFORK|CLONE_CLEAR_SIGHAND, exit_signal=SIGCHLD, stack=0x7e2f3e1e9000, stack_size=0x9000}, 88 <
unfinished ...>
4564  execve("/bin/sh", ["sh", "-c", "--", "curl -s -m 3 http://malware-c2.e"...], 0x7ffcbd613798 /* 13 vars */ <unfinished ...>
4563  <... clone3 resumed>)             = 4564
4564  <... execve resumed>)             = 0
4563  wait4(4564,  <unfinished ...>
4564  access("/etc/ld.so.preload", R_OK) = -1 ENOENT (Aucun fichier ou dossier de ce nom)
4564  openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
4564  openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
4564  getcwd("/home/med740/tp3", 4096)  = 17
4564  openat(AT_FDCWD, "/dev/null", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
4564  newfstatat(AT_FDCWD, "/usr/local/sbin/curl", 0x7ffe184c7ce0, 0) = -1 ENOENT (Aucun fichier ou dossier de ce nom)
4564  newfstatat(AT_FDCWD, "/usr/local/bin/curl", {st_mode=S_IFREG|0755, st_size=155, ...}, 0) = 0
4564  vfork( <unfinished ...>
4565  execve("/usr/local/bin/curl", ["curl", "-s", "-m", "3", "http://malware-c2.example/gate.p"...], 0x5a2c55f8c6b8 /* 14 vars *
/ <unfinished ...>
4564  <... vfork resumed>)              = 4565
4565  <... execve resumed>)             = 0
4564  wait4(-1,  <unfinished ...>
4565  access("/etc/ld.so.preload", R_OK) = -1 ENOENT (Aucun fichier ou dossier de ce nom)
4565  openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
4565  openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
4565  newfstatat(AT_FDCWD, "/home/med740/tp3", {st_mode=S_IFDIR|0775, st_size=4096, ...}, 0) = 0
4565  newfstatat(AT_FDCWD, ".", {st_mode=S_IFDIR|0775, st_size=4096, ...}, 0) = 0
4565  openat(AT_FDCWD, "/usr/local/bin/curl", O_RDONLY) = 3
4565  execve("/usr/local/sbin/wget", ["wget", "-q", "-T", "3", "-O", "-", "http://malware-c2.example/gate.p"...], 0x563c21308750
/* 14 vars */) = -1 ENOENT (Aucun fichier ou dossier de ce nom)
4565  execve("/usr/local/bin/wget", ["wget", "-q", "-T", "3", "-O", "-", "http://malware-c2.example/gate.p"...], 0x563c21308750 /
* 14 vars */) = -1 ENOENT (Aucun fichier ou dossier de ce nom)
4565  execve("/usr/sbin/wget", ["wget", "-q", "-T", "3", "-O", "-", "http://malware-c2.example/gate.p"...], 0x563c21308750 /* 14
vars */) = -1 ENOENT (Aucun fichier ou dossier de ce nom)
4565  execve("/usr/bin/wget", ["wget", "-q", "-T", "3", "-O", "-", "http://malware-c2.example/gate.p"...], 0x563c21308750 /* 14 v
ars */) = 0
4565  access("/etc/ld.so.preload", R_OK) = -1 ENOENT (Aucun fichier ou dossier de ce nom)
4565  openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
4565  openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
4565  openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libuuid.so.1", O_RDONLY|O_CLOEXEC) = 3
4565  openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libidn2.so.0", O_RDONLY|O_CLOEXEC) = 3
4565  openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libssl.so.3", O_RDONLY|O_CLOEXEC) = 3
4565  openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libcrypto.so.3", O_RDONLY|O_CLOEXEC) = 3
4565  openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libz.so.1", O_RDONLY|O_CLOEXEC) = 3
```

Figure 27 – trace2.log (début) copié sur l'hôte : execve, openat du marqueur, chaîne sh → curl → wget



Figure 28 – Récupération de trace.log par scp (conservé en annexe)

6. Partie 4 – Synthèse : IOC, ATT&CK et règle YARA

6.1 Indicateurs de compromission (pyramide de la douleur)

IOC	Type	Niveau	Durabilité	Valeur
SHA-256 sample.bin	Hachage	Bas	Fragile (recompilation)	58fc0097247b5a77bfc2a540b2fbc8c60df7349cf24464cce87c3e2abf353a5c
SHA-256 sample_packed.bin	Hachage	Bas	Fragile (repaquetage)	9543717941e1990ce3884ca8184a39277180ed8f40013e1ecab3b34567eba645
malware-c2.example	Domaine	Moyen	Modérée	GET http://malware-c2.example/gate.php
/tmp/.sentech_sample_marker	Artefact hôte	Moyen	Modérée	fichier caché contenant la note et la clé
Chaînes FAUSSE_CLE_XOR_5f3a9c, SENTECH-TRAINING-SAMPLE-BENIN	Chaînes	Moyen	Modérée	dans le binaire (non empaqueté)
Balise HTTP + @reboot + processus résident	TTP	Haut	Durable	`T1071.001, T1053.003, T1564.001, T1106`

6.2 Cartographie ATT&CK

Comportement	Tactique	Technique
Écriture d'un fichier marqueur caché dans /tmp	Defense Evasion	`T1564.001 Hide Artifacts: Hidden Files and Directories`
Requête HTTP vers un domaine C2	Command and Control	`T1071.001 Application Layer Protocol: Web Protocols`
Entrée @reboot ajoutée au cron	Persistence	`T1053.003 Scheduled Task/Job: Cron`
Processus enfant résident, commandes via sh -c	Execution	`T1106 Native API ; T1059.004 Unix Shell`

Remarque : le tableau de l'énoncé place le fichier marqueur dans « Exécution / Impact ». Un fichier caché déposé dans /tmp correspond mieux à T1564.001.

6.3 Règle YARA

```
rule SenTech_Sample_Malveillant
{
  meta:
    author = "SOC SenTech"
    description = "Detecte l'artefact d'analyse (chaines C2 + marqueur + cle)"
    attack = "T1071, T1053.003"
    reference = "TP3 - analyse de malware"
  strings:
    $c2 = "malware-c2.example" ascii nocase
    $mark = "/tmp/.sentech_sample_marker" ascii
    $key = "FAUSSE_CLE_XOR_5f3a9c" ascii
```

```
    $note = "SENTECH-TRAINING-SAMPLE" ascii
  condition:
    uint32(0) == 0x464c457f and 2 of ($c2, $mark, $key, $note)
}
```

Fichier testé	Résultat	Commentaire
sample.bin	Match (4 chaînes trouvées)	attendu
sample_unpacked.bin	Match	identique à l'original
sample_packed.bin	Aucun match	les chaînes sont compressées : limite d'une règle sur chaînes
/bin/ls	Aucun match	pas de faux positif

Note d'exécution : yara traite tous les arguments sauf le dernier comme des fichiers de règles ; la commande de l'énoncé avec plusieurs fichiers produit donc une erreur. Les fichiers ont été scannés un par un.
Amélioration possible : ajouter une règle sur la signature UPX (« UPX! ») et le fragment « malware-c2.examp » pour couvrir la variante empaquetée.

```
med740@med740-VMware-Virtual-Platform:~$ cd ~/tp3
cat > detect_sample.yar <<'EOF'
rule SenTech_Sample_Malveillant
{
  meta:
    author = "SOC SenTech"
    description = "Detecte l'artefact d'analyse (chaines C2 + marqueur + cle)"
    attack = "T1071, T1053.003"
    reference = "TP3 - analyse de malware"
  strings:
    $c2 = "malware-c2.example" ascii nocase
    $mark = "/tmp/.sentech_sample_marker" ascii
    $key = "FAUSSE_CLE_XOR_5f3a9c" ascii
    $note = "SENTECH-TRAINING-SAMPLE" ascii
  condition:
    uint32(0) == 0x464c457f and 2 of ($c2, $mark, $key, $note)
}
EOF
yara detect_sample.yar sample.bin sample_unpacked.bin sample_packed.bin /bin/ls
yara -s detect_sample.yar sample.bin
sample.bin(1): error: non-ascii character
SenTech_Sample_Malveillant sample.bin
0x200f:$c2: malware-c2.example
0x2041:$mark: /tmp/.sentech_sample_marker
0x20d6:$mark: /tmp/.sentech_sample_marker
0x202b:$key: FAUSSE_CLE_XOR_5f3a9c
0x205d:$note: SENTECH-TRAINING-SAMPLE
med740@med740-VMware-Virtual-Platform:~/tp3$
```

Figure 29 – Création de la règle et yara -s : chaînes trouvées dans sample.bin

```
med740@med740-VMware-Virtual-Platform:~/tp3$ cd ~/tp3
for f in sample.bin sample_unpacked.bin sample_packed.bin /bin/ls; do echo "== $f"; yara detect_sample.yar "$f"; done
== sample.bin
SenTech_Sample_Malveillant sample.bin
== sample_unpacked.bin
SenTech_Sample_Malveillant sample_unpacked.bin
== sample_packed.bin
== /bin/ls
med740@med740-VMware-Virtual-Platform:~/tp3$
```

Figure 30 – Scan fichier par fichier : sample.bin et sample_unpacked.bin matchent, sample_packed.bin et /bin/ls non

7. Recommandations de réponse pour le parc SenTech

- **Bloquer et alerter :** résolutions DNS et requêtes HTTP vers malware-c2.example, y compris GET /gate.php (proxy, RPZ DNS, pare-feu sortant).
- **Rechercher sur les hôtes Linux :** le fichier /tmp/.sentech_sample_marker ; les crontabs contenant une ligne @reboot pointant vers /tmp ; les processus orphelins nommés sample.bin (parent 1).
- **Scanner avec YARA :** déployer detect_sample.yar et la règle complémentaire sur la signature UPX pour les variantes empaquetées ; les hachages ne servent qu'à identifier l'échantillon connu.
- **Journaliser :** règles auditd sur l'exécution de crontab et l'écriture dans /var/spool/cron/crontabs ; corréler avec les connexions sortantes inhabituelles.
- **Durcir :** monter /tmp en noexec, limiter l'usage de crontab (cron.allow), filtrer les flux sortants par défaut.
- **Analyser :** tout futur échantillon dans ce laboratoire isolé, depuis un snapshot propre, en notant que strace peut masquer le comportement d'un programme setuid/setgid.

8. Partie 5 – Le cas Windows (PE) et bonnes pratiques

Hors ligne, pefile n'est pas installable ; l'inspection du format PE est faite avec objdump (binutils), qui lit aussi ce format. L'exécutable inspecté est ping.exe (binaire Windows bénin) copié sous le nom echantillon.exe.

Élément	Résultat
Signature / type	MZ ; PE32+ executable (console) x86-64 for MS Windows, 6 sections
Sections	Section .text, .rdata, .data, .pdata, .rsrc, .reloc
DLL importées (19)	15 api-ms-win-core-* (console, errorhandling, file, heap, libraryloader, localization, processenvironment, processthreads, profile, registry, rtlsupport, string, synch, sysinfo), IPHLPAPI.DLL, msvcrt.dll, ntdll.dll, WS2_32.dll
Indices de capacités	WS2_32.dll (sockets, réseau), IPHLPAPI.DLL (configuration IP), api-ms-win-core-registry (registre)

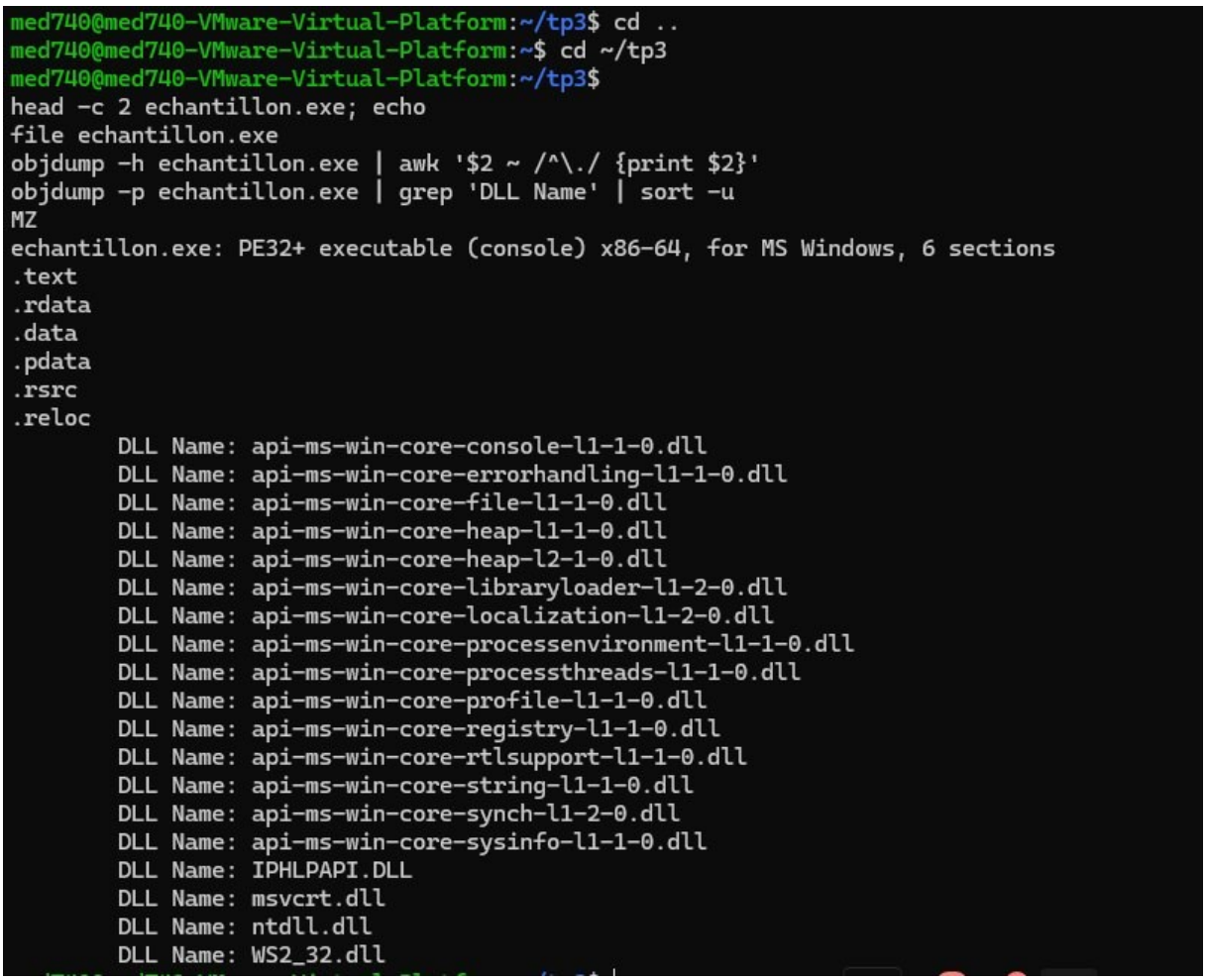


Figure 31 – Inspection du PE : signature MZ, sections et DLL importées

Trois règles d'or de manipulation d'échantillons

- **1. Isoler.** VM dédiée, réseau simulé sans Internet, snapshot propre avant chaque exécution, jamais sur un poste réel.
- **2. Défanger avant tout transfert.** Renommer le fichier (echantillon.exe.mal), l'archiver en ZIP protégé par le mot de passe « infected », ne jamais l'ouvrir par double clic.
- **3. Protéger la confidentialité.** Ne jamais téléverser un échantillon lié à un incident client vers un service public (VirusTotal et similaires partagent ce qui est envoyé) ; rechercher d'abord par empreinte.

9. Limites et écarts

- DNS d'INetSim remplacé par un répondeur Python (voir section 2) ; HTTP/HTTPS restent ceux d'INetSim.
- curl remplacé par un script wget : l'exécutable et le User-Agent diffèrent d'un vrai curl, le trafic HTTP est équivalent.
- strace en utilisateur masque l'écriture du crontab (bit setgid ignoré) : la persistance n'a été observée qu'avec sudo strace -u.
- La balise de la 2e exécution n'a pas fait l'objet d'une capture côté INetSim ; elle est documentée pour la 1re exécution (tcpdump, log INetSim, requêtes DNS).
- Le nombre brut de chaînes (100 contre 133) n'est pas un indicateur d'empaquetage fiable ici ; l'entropie, upx -t et le dépaquetage le sont.
- Fichiers conservés en annexe (hôte Windows) : trace.log (1re exécution) et trace2.log (2e exécution).